

UPPSALA
UNIVERSITET

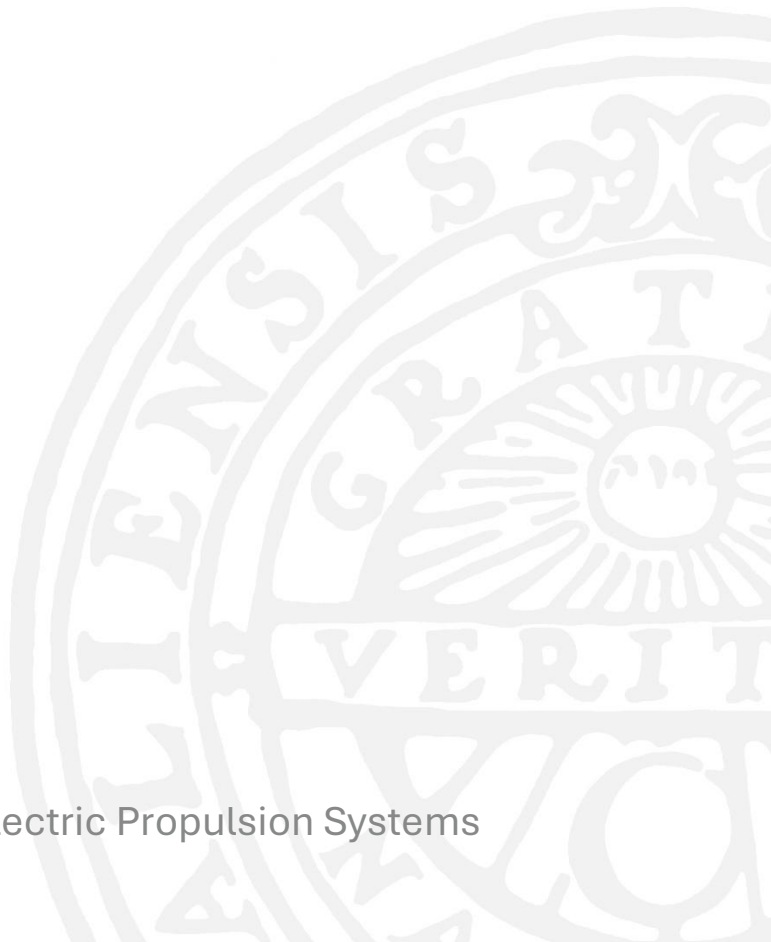
ELEKTRO-MEF; 26003

Degree project 30 credits

May 2026

Development of an electrical system prototyping tool to support the transition towards electric propulsion systems in aviation

Maximilian Schimmel



Master's Programme in All-Electric Propulsion Systems

UPPSALA UNIVERSITET

Development of an electrical system prototyping tool to support the transition towards electric propulsion systems in aviation

Maximilian Schimmel

Abstract

The transition towards electric and hybrid-electric propulsion in aviation increases the need for rapid, structured development of aircraft electrical system designs. In the studied industrial context at The Rainforest, Saab's innovation hub, early-stage electrical design work was mainly carried out in manually maintained Excel sheets. While initially practical, this workflow became difficult to scale as system complexity increased, leading to problems with collaboration, version history, system overview, data consistency, and usability for new users.

This thesis investigates how a dedicated Electrical System Design Tool (ESDT) can better support early-stage aircraft electrical design than the existing spreadsheet-based workflow. The work combined a literature review with observations and discussions at The Rainforest to identify workflow problems, derive requirements, compare concept directions, and develop a prototype.

The resulting ESDT is a browser-based client-server prototype built around a shared structured data model with linked graph and table views, explicit version handling, and controlled multi-user access. It was evaluated through a representative UAV-scale electrified propulsion architecture demonstration case, representing a turbine-generator replacement scenario.

The results show that the prototype can support the transition from early design definition to a more structured and connected system representation. It reduces reliance on manual data duplication, improves overview, and clarifies version and collaboration handling. The thesis does not demonstrate full industrial readiness, but provides prototype-level evidence that a lower-overhead, structured tool can address key limitations of the previous Excel-based workflow.

Faculty of Science and Technology

Uppsala University, Place of publication: Uppsala

Supervisor: Mattias Lindgren Subject reader: Jennifer Leijon

Examiner: Valeria Castellucci

Popular Scientific Summary

Electric and hybrid-electric aircraft are often seen as part of aviation's move toward lower emissions. Before such aircraft can be built, engineers first need to design the electrical systems that connect batteries, motors, converters, control units, sensors, cables, and protection devices into a working whole. This work starts early, when many ideas are still open and when the system may change several times before a stable design is reached.

This thesis studied that early design work in an industrial setting at The Rainforest, Saab's innovation hub. The Rainforest works in small teams and focuses on moving quickly from early ideas toward practical product candidates. In that kind of environment, engineers need tools that allow them to test and revise concepts without too much process overhead. For that reason, the early electrical system design work was mainly done in manually maintained Excel sheets. Excel was useful at first because it was familiar, flexible, and easy to change. It allowed engineers to start quickly without first setting up a complex engineering environment.

However, the same way of working became harder to manage as the system grew. Each new component, connector, cable, or interface had to be entered and updated manually. Some information appeared in more than one place, which meant that one change could require several separate updates. A simple example is connector or part-number information. If the same connector is used in different parts of the system, and that connector later changes, every occurrence has to be found and updated by hand. If one place is missed, the design can contain conflicting information. These kinds of inconsistencies may only become visible later, for example during procurement, build preparation, or testing.

The problem was therefore not that Excel was a bad starting point. It was that the workflow did not scale well once the electrical system became larger and more detailed. Collaboration became awkward, version history was difficult to follow, larger systems were hard to understand, and repeated manual data entry increased the risk of mistakes. Important design knowledge could also remain tied to the people who knew the spreadsheet best, instead of being clearly represented in the tool itself.

To understand this problem more broadly, the project combined observations from industry with a literature review. The review covered early-stage electrical system design tools, system modelling, graph-based system representation, design data management, versioning, and decision support. Together, these sources showed that there are many ways to model, analyse, and document system designs. However, they give less practical support for the step between rough early exploration and more mature engineering work. In practice, engineers often need to sketch, change, and test ideas quickly, but once a concept starts to become serious, the information also needs to become structured, shared, and reusable.

The project therefore investigated how early electrical system design work could remain flexible without losing structure as it matured. Based on the literature and the industrial workflow study, five main problem areas were identified: collaboration, version handling, system overview, data consistency, and usability for new users. These problem areas were then used to guide the development of a prototype tool.

The prototype was built as a browser-based Electrical System Design Tool, referred to as the ESDT. The main idea was to store the electrical system as one connected set of structured data, rather than as repeated information spread across different spreadsheet locations. The same underlying data could then be shown in different ways. A graphical view allowed the user to see components and connections as a system, while table views allowed detailed information to be

edited in a more familiar format. This meant that engineers could move between overview and detail without maintaining separate copies of the same design information.

The ESDT also included functions for version handling, subsystem grouping, import and export, undo and redo, and controlled multi-user access. Version handling was important because early design work often involves alternatives, earlier states, and changes that may need to be revisited. Subsystem grouping and filtering were important because electrical systems can quickly become visually dense once all components and connections are added. Controlled multi-user access was included to make shared work clearer and reduce the need for purely manual coordination between users.

The prototype was evaluated by building a representative battery-electric propulsion architecture for an unmanned aircraft. The case was not meant to produce a final aircraft design or a fully optimised powertrain. Instead, it was used to test whether the ESDT could support realistic early-stage design work. The demonstration included a simplified fixed-wing UAV propulsion system with two battery packs, battery management, contactors, a pre-charge circuit, high-voltage distribution, an inverter, an electric motor, a propulsion control unit, low-voltage supply, and representative sensing functions. This made the case large enough to test the tool's ability to handle components, interfaces, wiring, grouping, and system overview.

The results showed that this type of tool can support early-stage electrical design more coherently than the earlier spreadsheet-based workflow. The prototype made it easier to see the system as a connected whole, while still allowing detailed information to be edited in tables. It reduced repeated data entry, made design history clearer, supported subsystem-focused views, and gave a more structured basis for collaboration. It also showed how early design information can be moved toward a more reusable form without forcing engineers directly into a heavy formal tool environment.

The project does not present a finished industrial product, and it does not prove long-term time savings or full deployment readiness. The evaluation was based on one representative demonstration case and prototype-level evidence. Even so, the work shows that the identified gap is real. Early electrical system design needs tools that are light enough for fast concept work, but structured enough to keep growing system designs clear, consistent, versioned, and useful for later engineering steps. In that sense, the thesis contributes both a practical prototype and a clearer understanding of what is still missing between rough early ideas and mature aircraft electrical system design.

Preface

This thesis was carried out as part of the Master's Programme in All-Electric Propulsion Systems at Uppsala University, in collaboration with The Rainforest, Saab's innovation hub. The project gave me the opportunity to work on a real engineering workflow problem in an industrial setting, which was one of the main reasons the topic felt meaningful. It combined electrical system design questions with practical tool development in a way that was both technically interesting and directly relevant.

During the project, I gained insight into how early-stage electrical engineering work is carried out in practice, especially in the context of electrified aviation. The work also gave me the opportunity to deepen my knowledge in programming and software development for digital engineering tools. As the thesis is based on work carried out in an industrial environment, some details are presented in a simplified form.

I would like to sincerely thank my supervisor, Mattias Lindgren, for his guidance and support throughout the project. I would also like to thank my subject reviewer, Jennifer Leijon, for guiding me through the academic process and the thesis work. In addition, I would like to thank my examiner, Valeria Castellucci, for her support and for her role in the examination of this thesis. I am also grateful to my coworkers for their input, discussions, and feedback during the project. Finally, I would like to thank my family and friends for their encouragement and support throughout this work.

Linköping, May 2026

Maximilian Schimmel

Contents

Popular Scientific Summary 2
Preface 4
Contents 5
Table of figures 7
List of tables 7
Abbreviations 8
1. Introduction 9
1.1 Background and context 9
1.2 Problem description 10
1.3 Aim and objectives 11
1.4 Research questions 11
1.5 Scope 12
1.6 Thesis outline (what's in each chapter) 12
2. Literature review 13
2.1 Early-stage electrical system design 13
2.2 MBSE and system modelling 14
2.3 Graph-based system representation 15
2.4 Design data management and versioning 16
2.5 Decision support in early design 18
2.6 Positioning against existing tools 18
2.7 Chapter summary and consolidated research gap 20
3. Method 22
3.1 Method overview and research logic 22
3.2 Context and data collection approach 22
3.3 Problem identification procedure 23
3.4 Requirements derivation method 24
3.5 Concept generation and selection method 25
3.6 Implementation approach 26
3.7 Evaluation approach 26
4. Results 28
4.1 Identified workflow problems 28
4.2 Requirements and design objectives 29
4.3 Morphological chart 31
4.4 Identified design workflow 37
4.5 ESDT Prototype description 37

4.6 Demonstration case and evaluation outputs 46
4.7 Requirement coverage assessment 53
5. Analysis and Discussion 56
5.1 Discussion focus and evaluation framing 56
5.2 Addressing the research gap 56
5.3 Addressing the five identified workflow problems 57
5.4 Relation to literature 59
5.5 Limitations and validity considerations 59
5.6 Future work 60
5.7 Ethical aspects 60
5.8 Economic, environmental, and societal sustainability aspects 61
6. Conclusions 63
6.1 Answers to the research questions 63
6.2 Main contributions 63
6.3 Final conclusion 64
6.4 Future work 64
7. Bibliography 65
8. Appendix 68
8.1 AI usage statement 68

Table of figures

Figure 1 An autonomous unmanned aircraft system designed by The Rainforest [5] 10
Figure 2 Thesis project development flow 12
Figure 3 Primary user interface paradigm morphological chart 32
Figure 4 Core data representation and single source of truth morphological chart 32
Figure 5 System structuring and grouping morphological chart 33
Figure 6 Collaboration and concurrency control morphological chart 33
Figure 7 Edit recovery and change safety morphological chart 34
Figure 8 Version and variant management morphological chart 34
Figure 9 Navigation and scalability for large systems morphological chart 35
Figure 10 Extensibility and schema evolution morphological chart 35
Figure 11 External editing and analysis exchange morphological chart 36
Figure 12 Access and platform strategy morphological chart 36
Figure 13 High-level design workflow for early-stage electrical system design work 37
Figure 14 System-level overview of the ESDT 38
Figure 15 Data model and version scoping used by the ESDT 38
Figure 16 Global login view with entry to the admin page 39
Figure 17 Admin page for user and permission management 40
Figure 18 Work Tree concepts and available actions 40
Figure 19 Work Tree view showing the work history 41
Figure 20 Editor workspace with the graph view and a node sheet open at the same time 41
Figure 21 Graph view showing an example electrical system design 42
Figure 22 Graph-based connection creation through direct drag interaction 43
Figure 23 Multi-edit modal used to apply shared property changes across a graph selection 44
Figure 24 Table-focused editor view 45
Figure 25 Two-user edit-control flow 45
Figure 26 Cell-level edit block in the tabular view 46
Figure 27 Major propulsion architecture components added 47
Figure 28 Major components after interface and connector definition 48
Figure 29 Extended system design with auxiliary and supporting components 49
Figure 30 Final wired and bundled electric powertrain validation case 50
Figure 31 Grouping and filtering of subsystem content 51
Figure 32 Graph-to-table highlighting with installation and procurement-related fields 52
Figure 33 Bus close-up showing bundled wires 52

List of tables

Table 1 Existing tools in relation to the identified workflow problem 20
Table 2 Requirements and design objectives for the ESDT 30
Table 3 Mapping between identified workflow problems and related requirements 31
Table 4 Requirement coverage assessment matrix 54

Abbreviations

Abbreviation	Meaning
AB	Aktiebolag
AI	Artificial Intelligence
API	Application Programming Interface
CAD	Computer-Aided Design
CSV	Comma-Separated Values
DC/DC	Direct Current to Direct Current
DSM	Design Structure Matrix
ESDT	Electrical System Design Tool
E/E	Electrical and Electronic
HTTP	Hypertext Transfer Protocol
MBSE	Model-Based Systems Engineering
PLM	Product Lifecycle Management
PostgreSQL	Open-source relational database management system
Saab	Svenska Aeroplan Aktiebolaget
SQL	Structured Query Language
SQLite	Embedded relational database management system
SysML	Systems Modeling Language
UAS	Unmanned Aircraft System
UAV	Unmanned Aerial Vehicle
WebSocket	Real-time bidirectional web communication protocol
XLSX	Microsoft Excel Open XML Spreadsheet format

1. Introduction

In the transition towards cleaner aviation, the electrification of propulsion systems has emerged as a promising pathway to reduce emissions and dependence on fossil fuels [1]. However, evaluating electric and hybrid-electric propulsion concepts places new demands on how aircraft electrical systems are modelled and sized. At Saab's innovation subsidiary The Rainforest, these systems are currently designed using manually maintained Excel sheets, where each component and connection must be added, updated, and checked in multiple places. This workflow is time-consuming, difficult to scale to more complex system designs, and prone to human error, which limits the ability to rapidly explore new propulsion layouts and energy storage concepts. To better support this exploratory work, there is a need for a dedicated design tool that can represent electrical systems in a modular way, keep component and connection data consistent, and provide a clear visual overview of the system. This project aims to develop such an electrical system design tool for The Rainforest and demonstrate its usefulness by applying it to a representative UAV-scale electric propulsion architecture used as a turbine-generator replacement case.

1.1 Background and context

Before presenting the project, its background and context are outlined in this subchapter.

1.1.1 Saab

Saab AB (Svenska Aeroplan Aktiebolaget) was founded in 1937 in Sweden with the primary aim of developing and manufacturing military aircraft for the Swedish Air Force. Over the following decades the company expanded and merged with other industrial actors, evolving from a pure aircraft manufacturer into a diversified aerospace and defence group [2].

Today, Saab is a global company headquartered in Sweden, supplying products and services ranging from fighter aircraft and radars to command and control systems and civil security solutions. Its operations span multiple business areas, including Aeronautics, Dynamics, Surveillance and Kockums, with tens of thousands of employees and facilities in several countries [3].

1.1.2 The Rainforest

The Rainforest is Saab's internal start-up and "lung for transformative innovation" [4]. It was created as an autonomous unit within Saab to amplify and accelerate disruptive ideas, combining the company's long tradition of engineering with start-up-like agility. Founded in 2022, The Rainforest operates as an internal start-up in Saab's Aeronautics business area, with the explicit mission to "defy the conventional" and explore technologies and markets that may shape the company's future offerings [4].

The Rainforest works in small, cross-disciplinary teams that follow a build, measure, learn approach, and aim to move quickly from early concepts toward product candidates that are ready, or almost ready, for industrialization by other Saab organisations, rather than remaining in long pre-studies or purely demonstrator-oriented work. This way of working makes it possible to explore and revise technical concepts quickly, but it also creates a mismatch with more conventional engineering tool workflows when the need is for rapid iteration with direct control over evolving system definitions. At the same time, it also increases the importance of tool support that can keep evolving system design data structured, visible, and reusable as the work

matures. Projects are aimed at both civil and military applications and emphasise modular, scalable and cost-efficient solutions that can be brought from concept to operational systems.

1.1.3 Autonomous UAV

The unmanned aerial vehicle (UAV), shown in Figure 1, is being developed within The Rainforest and provides the aircraft context for this thesis. It was chosen because it gives the project a realistic aircraft case instead of a purely theoretical example. The UAV itself is not the focus of the thesis, but serves as the background for the project.

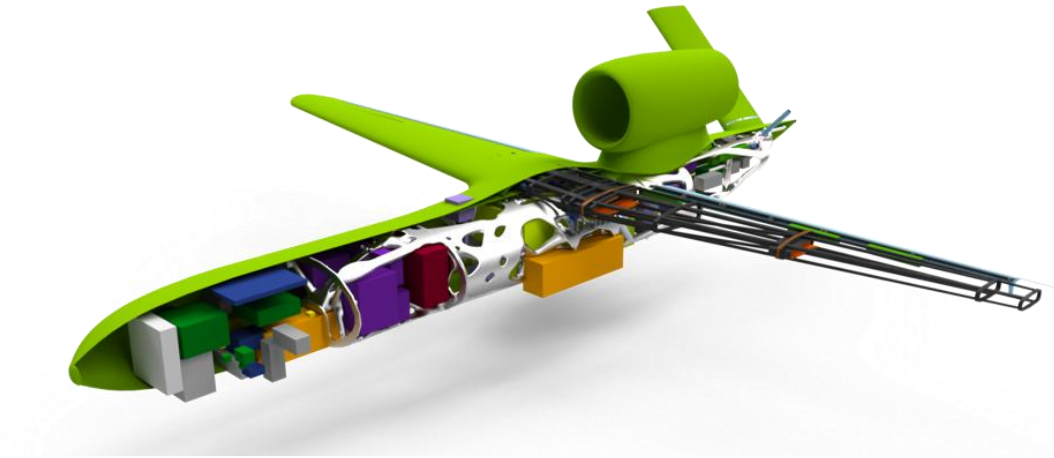


Figure 1 An autonomous unmanned aircraft system designed by The Rainforest [5]

1.2 Problem description

Electric and hybrid-electric propulsion are seen as important options for reducing emissions in future aircraft. To see if these concepts are realistic, engineers need to build and change electrical system designs many times. Today, this work at Saab's innovation unit The Rainforest is done mainly in manually maintained Excel sheets. Every new component and every new connection needs to be added in several places within the tools.

This manual way of working was a practical and fast way to get started in a small and agile development setting, where rapid iteration was more important than formal structure in the earliest phase. In that sense, Excel was not only a limitation but also a practical enabler of early work. However, as the system grew in size and detail, the same flexibility also made several workflow problems more visible. It became harder to maintain overview, harder to keep values consistent across tables, and easier for important design knowledge to remain tied to a small number of people rather than to the tool itself.

As electric propulsion architectures become more advanced, with several energy sources, power converters and safety paths, these limitations become a serious bottleneck. They make it difficult to explore new ideas quickly and to compare different concepts in a structured way. To support future electrical work including the transition towards electric propulsion, a more specialised tool is needed that can represent electrical systems in a clear and modular way, keep data in sync, and help engineers avoid obvious design errors.

Existing professional tools already support parts of electrical and system design work, but they are generally aimed at different stages, levels of formality, and engineering contexts. Heavy MBSE environments are strong in formal system modelling, traceability, and requirements linkage [6], [7]. Electrical CAD and E/E architecture tools are strong in schematic development, electrical system management, and, in several cases, continuity into later engineering stages [8], [9], [10].

Circuit simulation tools are strong in analysing electrical behaviour rather than supporting shared design workflow development [11].

The issue addressed in this thesis is therefore not that existing tools are incapable of supporting later development stages. Rather, in the studied context, the earliest iterative work benefited from a lower-overhead and more directly controllable way of working, which Excel provided only at the cost of weak structure, weak version handling, and repeated manual synchronisation. The need was therefore for a tool that could preserve more of that early flexibility while keeping the data structured, shared, versioned, and reusable as the work matured into procurement, production and testing.

1.3 Aim and objectives

The goal of this project is to develop a dedicated electrical system design tool that addresses key limitations of the current Excel-based workflow used at The Rainforest. The tool should store components and connections in a consistent way, keep their data synchronised, and offer both a tabular view and a graphical view of the system. It should help the user produce, edit, review, and test electrical designs in a more coherent and traceable way.

To show that the tool is useful in practice, it is applied to a representative UAV-scale electric propulsion architecture used as a turbine-generator replacement case. This case is used to demonstrate how the prototype supports early-stage electrical system design and alteration, in the studied industrial context.

In this thesis, the prototype developed in the project is referred to as the Electrical System Design Tool (ESDT). The ESDT is the specific software prototype developed to address the limitations of the current Excel-based workflow at The Rainforest. The name ESDT is used throughout the thesis when referring to this developed system, rather than to electrical system design tools in general.

1.4 Research questions

The main research question can be stated as:

How can a dedicated electrical system design tool (ESDT) be developed to better support the design of aircraft electrical architectures for electric and hybrid-electric propulsion than the current Excel-based workflow at The Rainforest?

The sub research questions can be listed as:

1. What are the main problems and limitations of the current Excel-based workflow?
2. What information about components, connections, and interfaces must the ESDT manage, and how should it be structured to remain consistent as the system design evolves?
3. Which functions and views are needed to efficiently create, modify, review, and communicate early-stage electrical system designs?
4. When the ESDT is applied to a representative electrified propulsion architecture demonstration case, what evidence shows that it supports the required workflow steps and reduces typical sources of inconsistency compared with the current Excel-based approach?

1.5 Scope

The scope of this project is limited to the development of the ESDT as a standalone design tool that supports basic data handling and visualisation of aircraft electrical systems. The focus is on the definition of the data model, the core functions for creating and modifying components and connections, and the visual representation needed to understand and communicate the system design. The project also includes using the ESDT to build a representative UAV-scale electric propulsion case study, used as a turbine-generator replacement case, and comparing this workflow to the current Excel-based approach.

Several related areas are deliberately kept outside the scope. The project does not include detailed design of the powertrain or the final selection of specific commercial components. It does not cover integration of the ESDT into existing Saab software environments or the development of a complete software ecosystem around it. Detailed thermal, mechanical or structural analysis of the propulsion system or its components is also excluded. These aspects are left for future work and for follow-on projects that can build on the results of this thesis.

1.6 Thesis outline (what's in each chapter)

The general development flow for this project is shown in Figure 2.

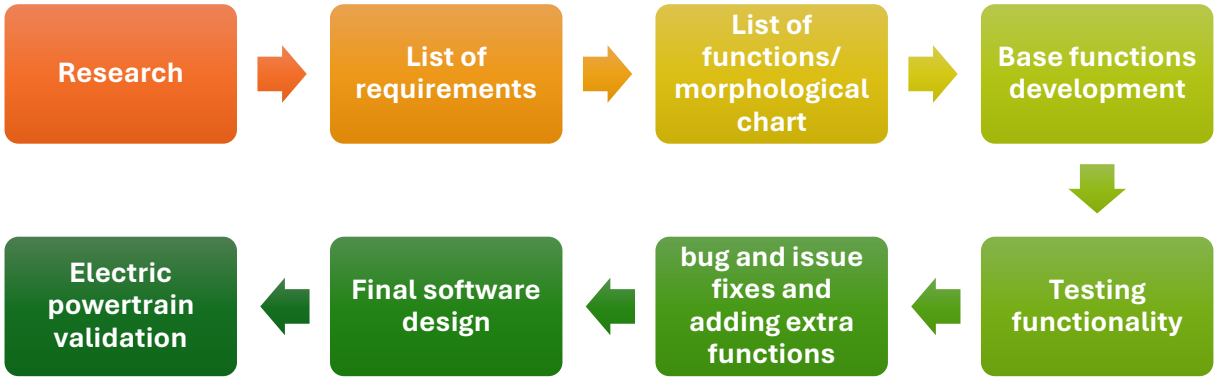


Figure 2 Thesis project development flow

The workflow shown in Figure 2 reflects the practical way the project was carried out, but in the thesis it has been structured according to a standard academic format. The literature review in Chapter 2 provides the background and research foundation for the study. Chapter 3 presents the method used to carry out the work, from problem investigation and requirement development to concept generation, implementation, and evaluation planning. Chapter 4 presents the resulting concepts, prototype, and validation results. Chapter 5 and 6 then discuss the results and present the final conclusions and future work.

2. Literature review

This chapter reviews the literature most relevant to the development of the ESDT as a practical tool for early-stage electrical system design work in electrified aircraft and related aerospace systems. The focus is on how early electrical system concepts are defined and compared, how design information can be represented as connected structured data, and how that data can remain consistent, traceable, and usable as the design evolves. The chapter is therefore organised into five topics:

1. conceptual electrical system design tools
2. MBSE and system modelling
3. graph-based system representation
4. design data management and versioning
5. decision support under uncertainty

The literature search was carried out mainly using Scopus. Search terms combined aerospace and electrical system terms with method and tool terms such as MBSE, SysML, system modelling, graph-based, Design Structure Matrix (DSM), traceability, configuration management, versioning, trade studies, and uncertainty. Papers were prioritised when they addressed early-stage system design definition or evaluation, structured system modelling, connected data representations for complex systems, or workflow-related topics such as traceability, version history, change impact, and collaboration. The selected literature was then used to identify the ideas and limitations that are most relevant to the thesis problem.

2.1 Early-stage electrical system design

Conceptual electrical system design tools mainly support early definition and comparison of aircraft power architectures. The discussion below focuses on what information they use, what they produce, and where they fall short as support for the workflow studied in this thesis.

2.1.1 Purpose, inputs, and outputs

Conceptual electrical system design tools are used to define and compare aircraft power architectures before detailed design data are available. The reviewed literature treats the aircraft power system as a set of interacting subsystems connected across several energy domains. Across the reviewed studies, meaningful early evaluation depends on an explicit architecture description and a consistent aircraft-level basis for comparison, rather than on isolated component sizing alone [12]. Early inputs typically include mission information, aircraft-level parameters, preliminary requirements, and assumptions about technologies, operating modes, and load behaviour [12], [13], [14]. Typical outputs include load profiles, sizing drivers, and aircraft-level comparison metrics such as mass, drag, power demand, or fuel-related effects, which are used to compare candidate architectures [12], [13], [14].

2.1.2 Evaluation approach and limitations

To compare candidate architectures efficiently, the reviewed tools usually balance speed and fidelity through staged evaluation. Candidate architectures may first be generated or filtered using coarse models and constraints, after which more detailed calculations are applied to a smaller set of shortlisted alternatives [14], [15]. This is necessary because conceptual design is characterised by incomplete data, large trade spaces, and strong dependence on assumptions about flight phases, operating modes, and failure or load-shedding cases [12], [13]. At the same time, the reviewed literature is stronger on modelling and performance evaluation than on

practical workflow support in early stage conceptual development and prototyping. It gives limited attention to collaboration, version history, variant handling, navigation of large systems, and maintenance of a single source of truth across users and views. A single source of truth means that each value is stored in one main place rather than repeated across several tables or views. This is important because early-stage system design work involves frequent changes, and repeated data quickly creates inconsistencies if one copy is updated while another is not.

2.1.3 Implications for this thesis

For this thesis, the main value of this literature lies in clarifying the data basis required for early system design work, rather than in providing a model for a full sizing or simulation tool. Even at conceptual stage, system architecture descriptions must capture components, interfaces, connectivity, and the context needed to compare alternatives in a consistent way [12], [13], [14]. The literature on early stage electrical system design therefore supports the need for structured system design data instead of fragmented spreadsheet work, and it supports the need for a representation that can be changed quickly without losing consistency.

2.1.4 Research gap summary

The reviewed conceptual-design literature provides methods for load estimation, system design screening, and aircraft-level comparison, but gives limited guidance on how a practical tool should support the day-to-day development of early system designs as shared and evolving design data. In particular, the literature is weaker on collaboration, persistent version history, structured handling of variants, navigation of large systems, and maintenance of a single source of truth across different views and users. Those limitations are central to the problem addressed by the ESDT in the agile workflow investigated in this thesis.

2.2 MBSE and system modelling

This section reviews Model-Based Systems Engineering (MBSE) literature that is relevant to early-stage system design work. It first focuses on how system elements, interfaces, and trace links are structured, and then discusses practical adoption issues and the relevance of a lighter MBSE approach in this thesis.

2.2.1 System modelling and traceability

The reviewed MBSE literature supports representing requirements, functions, interfaces, elements, and related analysis information in one connected and internally consistent model [16], [17], [18]. In practical terms, this means linking requirements to functions and system elements, defining interfaces explicitly, and maintaining consistency across abstraction levels. A repeated point is that system boundaries and exchanged flows should be defined explicitly from the start, since system design work depends on understanding what crosses interfaces and how different parts of the system interact [19], [20]. The literature also describes a progression from requirements and functions, through logical structure, toward physical implementation alternatives, even if authors use different terms for these stages [17], [19]. This is particularly relevant for early-stage electric propulsion architectures, where requirements, functions, interfaces, and physical components are tightly coupled across subsystems such as energy storage, power conversion, propulsion, control, and thermal management. In such systems, design decisions often affect not only electrical power flow, but also control logic, cooling needs, protection strategies, and safety-relevant interfaces [18], [21].

Another important theme is that one underlying system description should support several views. SysML-based and model-based approaches are used to keep structural, behavioural, and other

system views consistent instead of duplicating information across separate representations [22], [23]. The literature also supports linking requirements to system elements and using those links for traceability, change impact reasoning, and later verification planning where needed [16], [17], [24], [25], [26]. However, the reviewed work is less clear on how such links can be maintained in practical, low-overhead workflows over long periods of architectural change.

2.2.2 Practical adoption and lightweight MBSE

The literature also shows that MBSE adoption is limited by practical issues. Reported barriers include training needs, modelling effort, tool cost, lack of reusable assets, interoperability problems, and difficulty getting started [22], [23]. This is especially relevant in conceptual work, where many alternatives may be explored quickly and manual translation between system design descriptions and analysis tools can become a burden [18]. The literature does not suggest that every early-stage application needs a full enterprise-level SysML environment. Several papers support a lighter interpretation of MBSE, where the main value lies in explicit structure, linked information, formalised interfaces, consistent identifiers, and machine-readable data rather than in one specific toolchain [18], [23].

2.2.3 Implications for this thesis

In this project, MBSE was used more as guidance for how the ESDT should be structured than as a formal process to be followed in full. The main ideas taken from MBSE were that the system should be described in one coherent model, that interfaces and relationships should be made explicit, and that different views should remain connected to the same underlying data. At the same time, the ESDT still needed to stay light and practical enough for rapid early-stage work, without the higher threshold often associated with heavier MBSE environments.

2.2.4 Research gap summary

The reviewed literature shows the value of MBSE for structuring systems, linking requirements to design information, and maintaining consistency across views. However, it gives more guidance on model logic than on how a practical early-stage tool should stay easy to use while supporting large evolving system designs, multiple alternatives, and continuous change. These gaps are directly relevant to this thesis, which is concerned with practical design support rather than full-scale enterprise MBSE deployment.

2.3 Graph-based system representation

This section reviews literature on how engineering design information can be represented as connected structured data. It first discusses why relational representations are useful, when components, interfaces, and dependencies need to be stored and linked consistently, and then considers different representation approaches and their practical trade-offs.

2.3.1 Why relational representations matter

Engineering design data for complex systems are strongly relational. Components are connected through interfaces, requirements relate to functions and parts, and changes in one area can affect many others. The reviewed literature therefore gives strong support to graph-based and matrix-based representations for design work, since they make dependencies explicit and support analysis beyond simple document storage [27], [28], [29]. This is directly relevant to early-stage electrical system designs, where connectivity, interface compatibility, and change propagation are important alongside the definition of individual components.

A graph-based representation makes entities and their links directly visible. Nodes may represent components, functions, requirements, or other entities, while edges represent interfaces, flows, allocations, or dependencies [29], [30], [31], [32]. DSM-style representations capture similar information in matrix form and are especially useful when the main need is compact representation of dense dependencies, clustering, or decomposition analysis [27], [28]. Together, these representations support the idea that design data should be stored as connected structured information rather than as disconnected lists.

2.3.2 Representation choices and practical trade-offs

The literature also shows that different graph-based approaches serve different purposes. Simple node-edge graphs are useful for direct modelling and navigation. DSMs are strong for dense dependency analysis. Graph-based design languages and ontology-based approaches add stronger semantics, rules, interoperability support, and in some cases executable generation or formal reasoning [29], [31], [32], [33]. However, these richer approaches also increase modelling effort and maintenance demands. The reviewed literature therefore points to a practical trade-off: more semantics can improve queryability and interoperability, but it also makes the model harder to define, maintain, and scale. This trade-off is especially important in early-stage work. The literature supports explicit typing of entities, interfaces, and relations, but it also shows that representation choices must remain manageable in normal engineering use [28], [32]. For this thesis, that makes a moderate semantic level more suitable than a very heavy ontology-driven approach.

2.3.3 Implications for this thesis

Based on the reviewed literature, an ESDT for early-stage electrical systems should represent components, interfaces, and related entities through explicit relations rather than isolated records. The literature further indicates that a graph-oriented core representation is suitable for system structure and navigation, while tabular or matrix-like views remain useful for dense attribute work and dependency analysis. It also supports typed relations and basic queryability, but does not point to a need for the heaviest available semantic modelling approaches in this thesis context.

2.3.4 Research gap summary

The reviewed literature gives strong support for connected representations of design data, but gives less guidance on how a practical early-stage tool should balance navigation, clear semantics, low modelling effort, and ease of everyday use. In particular, it does not clearly show how one practical tool should combine graph-style modelling, structured data handling, and scalable use without becoming too formal or difficult to maintain. That research gap is directly relevant to this thesis.

2.4 Design data management and versioning

This section reviews literature on how engineering design information is managed as system designs evolve over time. It first addresses traceability, shared data, and versioning, and then discusses collaboration and change impact in relation to practical tool support.

2.4.1 Traceability, shared data, and versioning

A recurring problem in the reviewed literature is fragmentation of engineering information across documents, tools, and teams, which makes consistency, impact assessment, and collaboration difficult to maintain over time [34], [35], [36]. Traceability is therefore treated not only as the

existence of links between requirements and design information, but as a way to preserve alignment as system layout, interfaces, and related design decisions evolve [35], [37]. Several papers describe shared architectural structures that connect requirements, design elements, interfaces, and downstream activities, and use those links to support validation, change impact analysis, and coordination across development work [34], [35], [37].

The reviewed literature also shows that versioning of engineering models differs from ordinary code versioning. In MBSE and related environments, the main issue is not only textual change, but preservation of model meaning, structure, interfaces, and parameter relations as models evolve [38]. Repository-based model storage, controlled reference states, branches, commits, and model-aware comparison are therefore treated as important mechanisms for managing change in structured engineering data [37], [38]. The literature also indicates that code-centric mechanisms alone may be insufficient when semantic conflicts between model elements remain hidden [38].

2.4.2 Collaboration and change impact

In the reviewed literature, collaboration refers both to engineering-process coordination and to the technical management of shared system models, repositories, ownership boundaries, and edit states. Shared repositories, controlled interfaces, locking, decomposition into manageable ownership units, and explicit synchronisation between local and shared states are recurring collaboration mechanisms in the reviewed work [34], [35], [38]. Change impact analysis is also a major motivation for model-linked data management, because it helps identify what else may need to be reviewed or updated when one part of the system changes, and trace links and shared system structures make those dependencies more visible [34], [35], [38]. In addition, digital-thread-oriented work emphasises the value of preserving who changed what, when, and in relation to which version context, so that engineering data remain auditable over time [36], [38].

The reviewed work also shows that process rules, ownership definitions, and coordination procedures are not sufficient on their own. Without built-in support for shared system design data, version history, and visible links between related information, too much effort remains manual and coordination-heavy [34], [35], [37], [38]. This is especially relevant to the present thesis, because the industrial problem studied here is not only how to store data, but how to keep evolving system design information coherent and understandable during practical early-stage work.

2.4.3 Implications for this thesis

This part of the literature review has the most obvious bearing to the new ESDT prototype. It supports the need for one managed system design dataset instead of repeated manual synchronisation across several representations. It also supports persistent version history, shared visibility, explicit handling of change, and practical mechanisms for collaboration as central tool capabilities. For this thesis, the main implication is that system elements and their attributes should remain connected, consistently scoped, and revisitable over time, so that early system designs can evolve without losing coherence or traceability.

2.4.4 Research gap summary

The reviewed literature on design data management and versioning shows the importance of traceability, shared repositories, structured history, and model-aware change handling, but it does not yet provide a clear low-overhead solution for practical early-stage system design work. In particular, it remains weaker on how to maintain a single source of truth across multiple views

without recurring synchronisation work, how to handle variants cleanly during rapid iteration, and how to keep version history and traceability usable without adding too much process burden. This is where the literature most clearly leaves the practical workflow problem addressed here unresolved.

2.5 Decision support in early design

This section reviews literature on decision support in early design, with focus on trade studies, assumptions, and uncertainty. The aim here is mainly to provide broader context for why structured system design data matter, rather than to position this as the main implemented contribution of the thesis.

2.5.1 Relevance to early system design work

The reviewed literature shows that early design decisions have high impact and are made under uncertainty. At this stage, many system design alternatives are still open, but the available information is incomplete and later redesign can be costly [14], [39], [40]. The literature therefore includes deterministic trade studies, multidisciplinary design space exploration, risk-informed system design evaluation, and uncertainty-aware optimisation as different ways of comparing candidate concepts [14], [39], [40], [41], [42], [43]. Across these approaches, a common requirement is that alternatives, assumptions, scenarios, and metrics are defined clearly enough for results to be compared in a consistent way.

The literature also shows that trade studies need to be documented clearly. The compared alternatives, input assumptions, evaluation criteria, and reference case should be stated explicitly rather than left informal [44]. If this is not done, different disciplines may work with different assumptions, coordination becomes more difficult, and comparisons between alternatives become less reliable [40], [43], [44].

2.5.2 Implications for this thesis

In this thesis, literature on decision support in early design serves mainly as background rather than as a core implemented feature. The ESDT prototype is not intended to demonstrate a full trade-study, optimisation, or uncertainty-analysis environment. Its more immediate relevance is that later trade studies and architecture comparisons depend on system design data being structured, traceable, and linked to the assumptions and alternatives from which they arise. The thesis therefore addresses part of the foundation needed for more repeatable future decision support: a more coherent system representation and a more controlled way of capturing evolving alternatives.

2.5.3 Research gap summary

The literature offers strong methods for comparing alternatives under uncertainty, but says less about how assumptions and results should be managed in a practical collaborative workflow. For this thesis, the main conclusion is therefore not that the ESDT prototype must prove advanced decision support, but that practical early-stage design tools should make later comparison work more structured and repeatable.

2.6 Positioning against existing tools

There are already many tools that support parts of electrical and system design work but the tools that are easiest to identify around this area are usually aimed at somewhat different purposes

than the problem described in Chapter 1. A brief overview of the most relevant neighbouring tools is given in Table 1.

One relevant group is MBSE environments such as CATIA Magic / Cameo Systems Modeler and Capella / Arcadia. These tools are strong when the work depends on formal system models, traceability, and consistency between different system views [6], [7], [45], [46]. They are well suited for structured system modelling across several abstraction levels and for situations where requirements linkage is important. The need identified in this thesis is more practical and less formal than that. The focus here is early-stage electrical system design work, where changes may need to be made quickly and where the tool must be easy to use in day-to-day work. From that perspective, MBSE tools are clearly relevant, but they do not fit the identified problem especially well.

Another relevant group consists of electrical and E/E architecture tools such as Siemens Capital, Zuken E3.series, and Eplan. These are in some ways closer to the present problem because they work more directly with electrical system information and, in several cases, also support continuation into later engineering stages. Siemens Capital includes architecture-level E/E development and supports the allocation of functions and signals in a broader electrical design process [8]. Zuken E3.series supports work that can begin with functional design and continue toward detailed wiring and manufacturing-related outputs [10], [47]. Eplan Electric P8, together with Eplan Preplanning, also shows how structured electrical engineering data can be introduced early and then carried further into later engineering stages [9], [48]. Even so, these environments are generally aimed at broader enterprise E/E development, detailed schematics, harness work, manufacturing preparation, or plant and automation engineering. Because of that, they only partly match the need identified here. They are related to the same area, but they do not directly target the same lower-overhead, directly tailorable, and rapidly iterative workflow that motivated the ESDT prototype developed in this thesis.

LTspice represents another neighbouring category. It is useful for analysing circuit behaviour and component-level electrical performance [11]. That is not the main challenge addressed in this thesis. The problem here is not detailed circuit behaviour, but how electrical system architecture information can be created, changed, shared, and kept consistent during early-stage system work.

MATLAB System Composer is also worth mentioning as an adjacent tool. It is closer to the present problem than circuit simulation software because it supports system modelling, allocation, and links to later model-based analysis work [49]. At the same time, its main strength lies in connecting system design work to analysis- and simulation-oriented workflows rather than to procurement, manufacturing preparation, or build-oriented electrical documentation. That is somewhat different from the more practical problem addressed here.

Taken together, this points to a fairly specific gap. There are several established ways to model, analyse, and document complex electrical systems, and there are also mature tools for detailed electrical engineering work. What is less clearly supported is the earlier stage in between, where electrical system designs are still being defined and restructured, but where the information already needs to remain structured, shared, versioned, and reusable over time. This comparison is summarised in Table 1 and helps clarify the gap that motivates the work presented in the following chapters.

Table 1 Existing tools in relation to the identified workflow problem

Tool	Main strength	Why it is not a full fit for this thesis problem
CATIA Magic / Cameo Systems Modeler	Formal MBSE, SysML-based modelling, traceability, collaboration	Relevant as a structured system design environment, but more formal and process-heavy than the lower-threshold workflow identified here
Capella / Arcadia	Architecture-centred MBSE with method support and multiple views	Relevant for structured system modelling, but less directly focused on practical electrical system editing and redesigning
Siemens Capital	E/E architecture development and continuity toward downstream electrical design	One of the closest neighbouring tools, but broader and more suite-oriented than the specific workflow need identified here
Zuken E3.series	Functional design connected to detailed wiring and manufacturing documentation	Covers early work, but is more strongly aimed at detailed wiring, harness, and production-related engineering
Eplan Electric P8	Structured electrical schematic design and documentation	Strong in electrical engineering documentation, but less aligned with graph-first early system design exploration
LTspice	Circuit-level simulation and waveform analysis	Relevant as a boundary case, but does not address the system design workflow targeted in this thesis
MATLAB System Composer	System modelling linked to model-based analysis and simulation	Relevant adjacent tool, but stronger on simulation-connected workflows than on the practical niche identified here

2.7 Chapter summary and consolidated research gap

Overall, the literature points to several recurring needs in early-stage system design work. Conceptual electrical system design literature shows how systems can be defined and compared at aircraft level when information is still limited. MBSE literature supports explicit interfaces, linked design information, and consistency across several views. Graph-based and matrix-based representation literature supports treating system design data as connected relational information rather than disconnected records. Design data management literature highlights the importance of traceability, shared data structures, version history, and visibility of change impact. Decision-support literature shows why alternatives, assumptions, and scenarios should be structured if comparisons are to remain credible and repeatable.

The literature is strong on modelling methods, representations, and analysis approaches. What is less clear is how this should be turned into practical day-to-day support for early-stage electrical system design work, especially when the work changes quickly and still needs to remain structured and manageable. The stronger gap lies in practical support for early-stage system design work as an evolving team activity. The reviewed literature gives limited guidance on how engineers should move from fast exploratory design work toward structured and reusable data without losing speed, overview, or flexibility. It is also weaker on how collaboration, version

history, navigation of large system architectures, and maintenance of one coherent system definition should be handled in day-to-day conceptual work.

The consolidated research gap addressed by this thesis can therefore be stated as follows:

Existing literature and available tool categories provide several ways to model, analyse, or document system designs, but they give limited support for a lower-overhead workflow that supports rapid concept development and preliminary electrical system definition up to a level that can guide continued build, review, and iterative testing, while still keeping the resulting data structured, shared, versioned, and traceable over time.

3. Method

This chapter describes the approach used to address the research gap identified in Chapter 2. The gap concerns practical tool support for early-stage electrical design work that allows rapid definition and restructuring of system designs, while enabling a transition toward structured, versioned, and traceable system design data that can be matured over time. The method turns literature findings and observed workflow needs into requirements, uses them to select a concept, and evaluates the resulting prototype in a demonstration case.

3.1 Method overview and research logic

The method followed a linked progression from gap identification to prototype evaluation. The work proceeded through six main steps: literature review, industrial-context study and note-based data collection, problem identification, requirements derivation, concept generation and selection, and prototype implementation followed by demonstration-based evaluation. Literature and industrial-context observations were first used to define the practical workflow problem. These findings were then translated into requirements, used to compare alternative concepts, and carried into prototype development. The resulting prototype was then evaluated in a representative UAV-scale turbine-generator replacement case. The outputs of each step are reported in the Results chapter, and their implications and limitations are analysed in the Discussion chapter.

3.2 Context and data collection approach

The empirical part of the work was carried out in an industrial setting concerned with early-stage electrical system design work. The purpose was to understand how electrical systems were explored, documented, and discussed in practice, and to identify recurring difficulties relevant to the thesis problem through a practical, work-oriented approach.

The main empirical period at The Rainforest, Saab, lasted approximately four months, with the first four weeks being the most intensive. During this phase, near-daily observation, participation, and discussion were used to build an understanding of the existing workflow, the types of information handled during early-stage design, and the constraints shaping how the lead electrical systems engineer worked. The empirical material collected during this phase consisted mainly of notes from participation, observations, meetings, and discussions, rather than formal interview transcripts or recordings.

The observed activities covered several parts of the early-stage workflow, including component research, whiteboard-based system design reasoning, entry of design data into existing design tools and models, and related implementation and reverse-engineering work. The studied context was a relatively small and agile team in which individual members held broad responsibilities. In practice, most of the electrical system design work relevant to this thesis was carried out by one lead electrical systems engineer, who therefore became the primary observed user and the main source of workflow input. Discussions and clarifications also involved other engineering roles, which helped refine the identified problems and requirements from a broader practical perspective. In addition, the intended future use of such a tool was considered more broadly, since procurement-related users, build-related users, and less familiar users would also need to access and understand relevant parts of the design data even if they were not the primary designers.

The discussions were mostly informal and took place in connection with ongoing work, although more directed question-based discussions were also used when specific clarification was

needed. This made it possible to capture both spontaneous workflow issues and more deliberate reflections on tool needs.

The empirical work focused on early-stage system design and its transition to more structured design data. The purpose was not to produce statistically representative user research or to map the full organisation-wide process in detail, but to gather enough grounded evidence to identify recurring problems, derive credible requirements, and support the development of a realistic ESDT concept. In addition to identifying recurring workflow problems, the empirical phase also made it possible to define a high-level representation of the observed electrical system design workflow. This workflow description was not treated as a formal process model, but as a practical design input that clarified which stages of work the developed ESDT needed to support and how information moved between them. The resulting workflow diagram is presented in the Results chapter and was used as an additional guide for later interface and interaction design.

3.3 Problem identification procedure

The problem identification step converted the collected observations and discussions into a smaller set of recurring workflow problems that could be addressed through tool support. This step was carried out after the literature review had established the broader research gap, so that the empirical material could be interpreted in relation to that gap rather than as isolated observations.

The main working material for this step was a large notes document built up during the empirical phase. This document was reviewed repeatedly over time. Repeated review of the notes helped identify recurring patterns in where the current workflow created friction. The focus was on identifying practical obstacles that affected early-stage iteration, design clarity, data continuity, and collaboration.

Individual issues were first expressed as short problem statements. Similar statements were then grouped together when they reflected the same underlying difficulty. This thematic grouping was used to move from many scattered remarks to a smaller number of broader and more useful problem areas. The grouping was not performed through a formal coding procedure, but through iterative review and synthesis of the notes.

The consolidation of the final problems was guided mainly by three considerations: recurrence, impact, and relevance to the thesis scope. Recurrence referred to whether a difficulty appeared repeatedly across different situations, discussions, or examples. Impact referred to whether the difficulty clearly affected the speed, quality, or manageability of early-stage system design work. Relevance referred to whether the issue could reasonably be addressed through the type of tool support investigated in the thesis. Problems that were too narrow, too case-specific, or outside the scope of early-stage design support were not retained.

This process resulted in a final set of five key problems. These problems are presented in more detail in the Results chapter but can be broken down into: collaborative work, version history management, large system complexity, data consistency, usability for new users. In the method context, the important point is that they were not chosen as isolated opinions, but as a synthesised set derived from repeated review of empirical notes in combination with the research gap established in the literature review.

The resulting problem statements were then discussed with the lead electrical systems engineer and refined through further conversation with other roles familiar with electrical system design. This validation was informal and practical rather than formal. The goal was to check that the

wording was recognisable, that the identified problems matched the experience of intended users, and that the final problem set was focused on issues that were genuinely worth addressing. Through this back-and-forth process, the problem statements were honed before being used as input to the requirements step.

3.4 Requirements derivation method

The requirement set was derived from three main inputs: the literature review, the identified electrical system design workflow problems, and continued discussion with the lead electrical systems engineer and other relevant roles. The literature review provided method-level implications regarding what early-stage system design support should enable, such as structured representation, change handling, data continuity, and support for comparison and traceability. The empirical work grounded these implications in observed practical needs. The requirements were therefore intended to connect theory and practice rather than to reflect only one source.

The derivation process was performed by reviewing the five consolidated problems and translating them into statements of what the tool must or should support. In parallel, findings from the literature and observations of selected neighbouring tool categories, including MBSE environments, E/E design tools, and circuit simulation tools, were used to identify additional needs that were important for the intended use context even when they were not expressed directly in the same wording by the lead electrical systems engineer or other relevant contributors. This made the requirement set both practical and forward-looking.

The requirements were written at a level suitable for two later uses. First, they had to be clear enough to support comparison of alternative concepts during concept selection. Second, they had to be concrete enough to support later prototype evaluation through requirement coverage. For this reason, the requirements were formulated as functional or qualitative support statements rather than as highly technical implementation constraints.

To keep the requirement set structured, the requirements were organised into categories. The categories used were basic design function, basic procurement function, basic build function, data handling, design function, future, compatibility, and usability. These categories were used to make the set easier to review and compare. They also helped ensure that the requirements reflected more than only the immediate design activity, since the intended tool needed to support both engineering work and communication of system design data to other users.

A distinction was also made between hard and soft requirements. Hard requirements were requirements considered necessary for the tool to address the core need of the thesis and to provide a meaningful prototype within scope. These were typically functions that could be stated clearly and assessed directly, such as storing and editing system data, maintaining version history, supporting concurrent access, or preserving a single source of truth. Soft requirements were desirable requirements that were more subjective in nature or harder to measure directly, such as intuitiveness, ease of orientation for people who are not familiar with the project. These were still important, but they were treated differently in the later evaluation because they relied more strongly on judgement and qualitative feedback.

The requirement set was developed iteratively rather than in one single step. The main structure came from review of notes, synthesis of the identified problems, and discussion with the lead electrical systems engineer, as well as other relevant roles. Some requirement formulations were also clarified during concept development and prototype work as the practical implications of the problem set became more explicit. This did not change the overall direction of the work, but it

helped sharpen the wording and make the requirements more usable for concept comparison and evaluation.

The final requirement set provided the bridge between the empirical problem identification and the later design decisions. In this way, the concept generation and prototype implementation were not based only on intuition, but on a requirement set that could be traced back to both observed workflow needs and the broader literature-based research gap.

3.5 Concept generation and selection method

After the requirements had been defined, alternative solution concepts were generated and compared. The idea behind this step was to avoid moving directly from problems and requirements to implementation without first examining different ways of structuring the tool. A morphological chart was used as the main concept generation method because it allowed major design decisions to be separated into dimensions and compared systematically.

The morphological chart was built around ten parameters that were judged to be central to the tool concept. These were:

1. primary user interface paradigm
2. core data representation and single source of truth
3. system structuring and grouping
4. collaboration and concurrency control
5. edit recovery and change safety
6. version and variant management
7. navigation and scalability for large systems
8. extensibility and schema evolution
9. external editing and analysis exchange
10. access and platform strategy

For each parameter, several alternative options were defined and compared.

The chart was used to force explicit design choices instead of leaving core decisions implicit. For example, the method required comparison between a purely tabular interface, a purely graphical interface, and a hybrid interface; between ad hoc graph-style data handling and a more structured relational model; between file-based and centralised collaboration approaches; and between simple manual recovery and more deliberate undo, versioning, and branching strategies. In this way, the concept generation step covered both user interaction and underlying data management.

The alternatives were compared mainly through reasoned discussion against the requirement set rather than through formal scoring. This was done deliberately. Many of the design choices involved trade-offs that were strongly qualitative, and a numerical scoring model would have introduced a level of precision that was not well supported by the available evidence. Instead, each option was judged in terms of requirement fit, practical suitability for early-stage system design work, and feasibility within thesis scope.

Requirement fit was used as the main comparison principle. Options that clearly conflicted with hard requirements or with the central research gap were excluded. Beyond that, the comparison also considered whether an option would support intuitive use, future scaling, and the transition from rapid early design work to more structured data handling. Implementation feasibility was also considered, since the selected concept had to be realistic to prototype within the scope of a master's thesis.

The final concept was chosen through a structured comparison of the alternatives. The selected concept is reported in the Results chapter together with the chart and the main selection rationale. In method terms, the important point is that the concept was not chosen ad hoc. It was selected through an explicit comparison of alternatives across the main parameters that shape how such a tool would function.

3.6 Implementation approach

The selected concept was realised as an ESDT prototype intended to test feasibility rather than to serve as a complete production-ready engineering platform. The implementation of the ESDT therefore focused on the minimum set of capabilities needed to examine the central thesis contribution: support for rapid early-stage system definition and revision while maintaining more structured, shared, and traceable system design data.

The implementation of the ESDT followed an iterative development approach. Work began by establishing a basic internal structure for representing system elements and their relations, together with an interaction pattern suitable for early-stage editing. After that, the ESDT prototype was refined through repeated cycles in which functions and data structures were adjusted in response to the requirement set and the intended demonstration case. This made it possible to test design decisions progressively rather than fixing all implementation choices in advance.

A central implementation objective was to combine flexible early-stage editing with structured data consistency. For this reason, the ESDT prototype was developed around a structured internal data model and an interface approach that could support both system overview and detailed editing. This reflected the broader method logic of the thesis, in which whiteboard-like iteration needed to be combined with a transition toward more explicit and reusable design data.

The implementation also addressed collaboration, recovery, and version handling at prototype level, since these aspects were part of the identified workflow needs and requirement set. The purpose was not to implement a full industrial platform, but to realise these functions far enough to allow meaningful evaluation in the demonstration case.

The implementation scope was intentionally limited to the functions required for concept demonstration and requirement assessment within thesis scope. Broader lifecycle integrations and full production-system support were outside the scope of the ESDT prototype and were not necessary for answering the research question.

3.7 Evaluation approach

The evaluation was structured around a representative demonstration case based on a UAV-scale electric powertrain architecture used as a turbine-generator replacement case. This type of case was chosen because it matched the application domain of the thesis while remaining limited enough to demonstrate the ESDT clearly. It also provided a suitable basis for exercising both initial system creation and later restructuring, which were central to the research gap addressed by the work.

The evaluation was not designed as a statistical comparison with existing tools or as a formal usability study. It was intended instead to determine whether the ESDT could demonstrate the required functions in a coherent use case and address the identified needs at a level appropriate for a thesis prototype. The evaluation therefore combined direct functional demonstration with requirement-based assessment and limited qualitative feedback for the more subjective requirements.

The demonstration case was designed to exercise the requirement set through a sequence of concrete tasks. These tasks included creating a mock powertrain architecture by adding components, connections, and attributes; editing the system design after creation; grouping or clustering parts of the system; filtering and compiling procurement-related lists; exporting structured system design data for external use; demonstrating undo and redo; saving and revisiting versions; handling branch-based variants; showing simultaneous access from multiple users; and observing how conflicting changes were handled. In addition, the case was used to assess whether the system representation was understandable for users beyond the main design role, particularly for build-related or less initiated users.

Requirement coverage was assessed through a requirement coverage table. For the more objective hard requirements, the assessment was based mainly on whether the corresponding function could be demonstrated directly in the ESDT prototype. For the more subjective soft requirements, the assessment was also supported by qualitative judgement and user feedback, for example regarding intuitiveness, ease of orientation, or the usefulness of the representation for non-primary users. The final assessment therefore distinguishes between direct prototype demonstration, implemented architectural support, and feedback-supported fulfilment where relevant.

The evaluation followed directly from the earlier requirements and design steps. The same requirements that had been derived from literature implications, empirical observations, and user discussions were used first to guide concept selection and later to structure prototype evaluation. This created a continuous chain from research gap to empirical problem identification, from problems to requirements, and from requirements to demonstration-based assessment.

The evaluation had clear delimitations. It was based on a single representative demonstration case and did not aim to prove long-term industrial impact, full-scale deployment readiness, or superiority over other tools through controlled comparison. The compatibility requirement was assessed separately from the demonstration case itself, since operating system support is a platform characteristic rather than a powertrain modelling task. Similarly, future-oriented requirements such as transition toward production-oriented use or support for larger-scale growth were assessed mainly through prototype structure, data handling approach, and demonstrated export or extension capability rather than through full downstream deployment.

Within these limits, the evaluation approach was suitable for the thesis purpose. It allowed the ESDT prototype to be assessed in relation to the actual problems and requirements that motivated its development, while keeping the scope realistic for a master's thesis prototype.

4. Results

This chapter presents the identified workflow problems, the derived requirements, the selected concept, and the resulting prototype. The results are based mainly on observations and discussions with the lead electrical systems engineer in the studied industrial context.

4.1 Identified workflow problems

The initial observations and discussions in the industrial setting identified five recurring workflow problems in the existing tool environment. These problems concern:

1. Collaborative work
2. Version history management
3. Large system complexity
4. Data consistency
5. Usability for new users

Together, they describe the main limitations of the current workflow for early-stage electrical system design work.

4.1.1 Collaborative work

In the existing tool environment, simultaneous work on the same system design within the main design document was difficult. Although most of the electrical system design work was carried out by one lead electrical systems engineer, other engineers also needed to review, add to, or modify parts of the design at certain stages. In practice, this was handled through a manual blocking approach, where one engineer indicated that the system was currently being edited and others refrained from making changes at the same time. This may have been manageable in a small setting, but it still created coordination overhead and became increasingly restrictive when additional input, integration, or later reuse of the system design data was needed. The limitation therefore affected both direct simultaneous editing and the broader ability to work with shared evolving design data.

Impact:
This slows collaborative iteration and increases the risk of delayed updates, coordination overhead, and conflicting edits.

4.1.2 Version history management

Version history management was found to be split across two different practices in the current workflow. On one side, snapshots of system designs are saved manually as backups. On the other side, minor system permutations and sub-versions are recorded within the design document itself, for example by marking parts of the design as conditionally applicable. Together, these approaches create document bloat and make it difficult to distinguish between backup states, design revisions, and alternative system variants.

Impact:
This reduces traceability and makes it harder to compare alternatives, recover earlier states, and maintain a clear overview of design evolution.

4.1.3 Large systems complexity

As a system grows, the current tool becomes increasingly difficult to navigate. At small scale, the number of components and interconnections remains manageable, but at higher complexity the existing structure relies heavily on filtering functions to make the design usable. These filters are necessary, but they do not remove the underlying difficulty of understanding the overall system. The representation is effective for dense technical information, yet it often requires tedious backtracking of interconnects and repeated navigation across the design to understand subsystem relationships.

Impact:
This limits scalability and makes it more difficult to retain system overview as the system's size and complexity increase.

4.1.4 Data consistency

In the current workflow, some pieces of information are represented multiple times within the same design document. This creates situations in which a value is entered in one place but not in another, or updated in one location while remaining outdated elsewhere. A typical example is connector-related information such as part numbers, which may need to be repeated in multiple locations. When such information changes, each occurrence must be updated manually. If one occurrence is missed, inconsistencies are introduced into the design data.

Impact:
This increases the risk of mismatched information and creates additional rework and errors in production, when inconsistencies are discovered later in the design process.

4.1.5 Usability for new users

The current tool environment was adopted because no better alternative was available. It was originally developed for smaller systems and then gradually expanded as system complexity increased. As a result, functions were added over time in ways that introduced workarounds, special cases, and interface quirks. This makes it harder for new users to understand how the system is structured, how information is organized, and how to gain an overview of the design without prior familiarity with the workflow.

Impact:
This raises the entry threshold for new users and increases the risk of misunderstanding the system design or editing it incorrectly.

4.2 Requirements and design objectives

Based on the literature review and the identified workflow problems, a set of requirements and design objectives was defined for the ESDT. These requirements translate the observed needs of early-stage electrical system design work into a structured basis for subsequent concept generation and ESDT development. The resulting requirement set is presented in Table 2 and retains the original hard/soft classification.

4.2.1 Requirements table

The complete set of requirements is presented in Table 2.

Table 2 Requirements and design objectives for the ESDT

Requirement ID.	Requirement	Category	Hard or soft requirement
1	The tool must allow for storing and editing of electrical system design data. E.g.: components and their attributes, connections between components and their attributes.	Basic design function	Hard
2	The tool must allow for filtering and compiling lists of components and cabling for procurement.	Basic procurement function	Hard
3	The tool must allow the building crew to see and understand the system and how to build it.	Basic build function	Hard
4	Every attribute of a component or connection must have a single source of truth. (no need to change one parameter at multiple locations)	Data handling	Hard
5	Bundling and splicing of buses and similar connection structures should be handled in an intuitive way.	Design function	Soft
6	The tool should display the designed system in a way that an uninitiated observer can find their way around the system easily.	Design function	Soft
7	A system designed in this tool must have a way to extract the data so that it can be sent to an external user for editing or analysing.	Data handling	Hard
8	The tool should have backup and undo and redo functions	Design function	Hard
9	Changes must be tracked, and version history must be kept to uphold the possibility to revert back to previous versions.	Data handling	Hard
10	Multiple users must be able to access the data simultaneously, and any conflicting changes must be resolvable by the users.	Data handling	Hard
11	There must be the possibility to add new attributes or status checklists to the components and interconnects.	Design function	Hard
12	Components and interconnects should be clusterable into different categories.	Design function	Soft
13	The system should allow for a transition into a production management tool in the future.	Future	Soft
14	The tool should be modular to allow for scaling to larger projects and database formats in the future.	Future	Soft
15	The tool must run on all operating systems; Linux, Windows, macOS.	Compatibility	Hard
16	The tool should be intuitive and easy to use for users without requiring deep programming or electrical system knowledge.	Usability	Soft

4.2.2 Mapping between workflow problems and requirements

The identified workflow problems were not translated into isolated one-to-one requirements. Instead, each problem area gave rise to a group of related requirements. This relationship is summarized in Table 3.

Table 3 Mapping between identified workflow problems and related requirements

Identified workflow problem	Related requirement IDs
1) Collaborative work is difficult	9, 10
2) Version history management is needed	8, 9
3) Large systems are difficult to manage	5, 6, 11, 12, 14
4) Values do not all have a single source of truth	4, 7, 11
5) New users have a hard time understanding the system	3, 5, 6, 12, 16

4.3 Morphological chart

The morphological chart was developed from the five problem areas identified in Section 4.1 together with the requirement set for the proposed tool. The problem analysis showed that the current process has weaknesses in collaborative work, version and variant handling, large-system manageability, maintenance of a single source of truth, and usability for new users. The requirements then translated these issues into concrete needs, including multi-user access, conflict handling, structured version history, scalable system structuring, extensibility, cross-platform access, and support for procurement, build, and external analysis. On this basis, the main concept dimensions for the chart were defined.

A morphological chart is a concept generation method in which a complex design problem is decomposed into key parameters, and alternative solution principles are listed for each parameter. Rather than selecting a complete concept directly, the method supports a systematic comparison of different solution combinations and helps make trade-offs explicit.

In this thesis, the morphological chart is used to structure the ESDT. The final parameters are:

1. Primary user interface paradigm
2. Core data representation and single source of truth
3. System structuring and grouping
4. Collaboration and concurrency control
5. Edit recovery and change safety
6. Version and variant management
7. Navigation and scalability for large systems
8. Extensibility and schema evolution
9. External editing and analysis exchange
10. Access and platform strategy

Together, these parameters capture the design choices that most strongly affect how well the tool addresses the identified workflow problems and requirements, and they provide a transparent basis for the subsequent concept selection.

4.3.1 Primary user interface paradigm

The primary user interface paradigm defines how users mainly view and edit the electrical system design. This was a central parameter because the tool must support both detailed attribute editing and intuitive understanding of system structure, especially for new users and larger systems. A hybrid tabular and 2D graph interface was therefore selected, since it combines efficient list-based work with a clearer visual overview of components and connections. Although this introduces more interface complexity than a purely tabular or purely graphical solution, it provides the best overall balance for the intended use case.

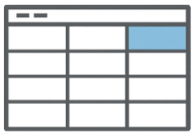
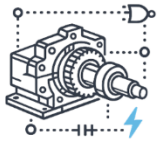
	Option 1	Option 2	Option 3	Option 4
Name	**Pure tabular editor**	**Pure 2D graph editor**	**Hybrid tabular + 2D graph interface**	**CAD-embedded interface**
Picture				
Description	All work is performed in spreadsheet-like tables.	All work is performed directly in a node-link diagram.	Tables and graph views are used together and remain synchronized.	The architecture tool is embedded in, or tightly coupled to, a CAD environment.
Advantages	Familiar data entry; strong for procurement lists and bulk editing.	Strong topology overview; intuitive for following system connections.	Combines strong data editing with strong system overview; supports both experts and new users.	Strong geometry context; can align directly with later design tooling.
Disadvantages	Weak system overview; difficult for uninitiated observers to interpret.	Dense attribute editing becomes inefficient; tabular export thinking is weaker.	Higher interface complexity than a single-view tool.	Scope becomes much larger than needed for early-stage architecture work.
Requirement fit	**Medium**	**Medium**	**High**	**Low**
Selected	No	No	**Yes**	No

Figure 3 Primary user interface paradigm morphological chart

4.3.2 Core data representation and single source of truth

The parameter for core data representation concerns how components, connections, and related information are stored at a fundamental level. It was particularly important because the identified problems included duplicate values, weak traceability, and lack of a single source of truth. A structured relational model with explicit entities and unique attribute ownership was selected, since it offers the strongest support for consistency, filtering, validation, and future extensibility. This approach is more demanding to define initially than an ad hoc structure, but it best satisfies the long-term requirements of the tool.

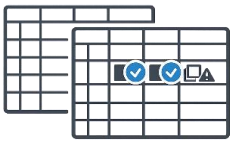
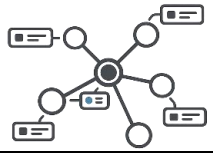
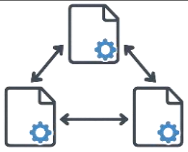
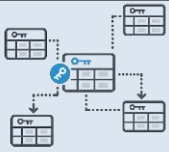
	Option 1	Option 2	Option 3	Option 4
Name	**Flat spreadsheet with duplicated values**	**Graph objects with ad hoc attributes**	**Separate subsystem files linked together**	**Structured relational model with explicit entities and unique attribute ownership**
Picture				
Description	Components and connections are stored in flat sheets, often with repeated values.	Architecture is stored mainly as nodes and edges with loosely structured attribute fields.	Each subsystem is maintained separately and linked through references.	Components, connections, groupings, and related attributes are stored in explicit structured entities.
Advantages	Simple to start with; easy to inspect manually.	Flexible for conceptual modeling; aligns with visual network thinking.	Can mirror organizational decomposition and local ownership.	Best support for single source of truth, filtering, export, validation, and future extension.
Disadvantages	Duplicate values easily emerge; traceability and consistency degrade over time.	Attribute ownership can become unclear; external analysis and structured export are harder.	Cross-subsystem filtering, consistency, and version handling become more difficult.	Requires more deliberate schema design than an ad hoc file approach.
Requirement fit	**Low**	**Medium**	**Medium**	**High**
Selected	No	No	No	**Yes**

Figure 4 Core data representation and single source of truth morphological chart

4.3.3 System structuring and grouping

The system structuring and grouping parameter concerns how the tool represents intermediate architectural structure, such as subsystems, bundles, and splice-like groupings. This was important because the tool must support clustering, build understanding, and navigation in larger systems. Explicit grouping entities were selected because they provide much stronger structural support than simple tags, while avoiding the excessive detail of a full physical harness model. The selected approach introduces some additional modeling effort, but it is well aligned with the early-stage scope of the thesis.

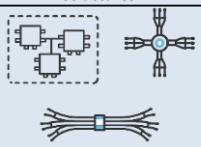
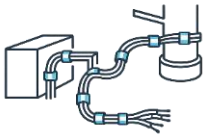
	Option 1	Option 2	Option 3	Option 4
Name	**Direct point-to-point model only**	**Category tags only**	**Explicit grouping entities for subsystems, bundles, and splice-like structures**	**Full physical harness and routing model**
Picture				
Description	The model contains only components and direct connections.	Components and connections can be tagged, but not grouped through explicit system objects.	The model includes explicit constructs for grouping and structuring the architecture at intermediate levels.	The model represents detailed physical routing, geometry, and manufacturing-level harness structures.
Advantages	Simple base model; low initial complexity.	Easy to add and useful for lightweight filtering.	Supports clustering, bundling, subsystem views, and more intuitive system interpretation.	Strongest support for downstream physical design and manufacturing preparation.
Disadvantages	Weak support for bundling, clustering, and system-scale understanding.	Tags alone are often too weak for handling bundles, splices, and structural relations.	More modeling decisions are required from the user than in a completely flat model.	Too detailed and heavy for the early-stage scope of this thesis.
Requirement fit	**Low**	**Medium**	**High**	**Medium**
Selected	No	No	**Yes**	No

Figure 5 System structuring and grouping morphological chart

4.3.4 Collaboration and concurrency control

The collaboration and concurrency control parameter concerns how multiple users can access and modify the same system design model safely. This parameter was critical because limited support for collaborative work was one of the main problems identified in the current process. A central multi-user data store with live updates and explicit conflict detection was selected, since it most directly enables concurrent work while preserving shared visibility and data integrity. Other approaches were judged less suitable because they either rely too heavily on manual coordination or introduce a technical workflow that is less accessible to the intended users.

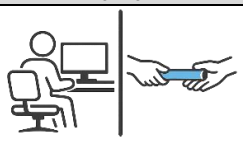
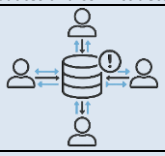
	Option 1	Option 2	Option 3	Option 4
Name	**Single editor or turn-taking workflow**	**Shared cloud files with manual conflict handling**	**File-based Git workflow**	**Central multi-user data store with live updates and conflict detection**
Picture				
Description	Only one person edits at a time, or users coordinate manually.	Multiple users access shared files and resolve overlaps manually.	Architecture data is stored as files and collaborated on through version control workflows.	Users work against a shared central data source with explicit conflict handling.
Advantages	Very simple to explain and implement.	Low setup threshold if users already work with shared files.	Strong history and branching mechanisms for technical file workflows.	Directly addresses collaborative editing, conflict handling, and shared visibility.
Disadvantages	Does not solve the identified collaboration problem.	Conflicts remain user-managed and error-prone.	Too technical for many intended users; weak fit for live collaborative editing.	Requires deliberate concurrency logic and user feedback mechanisms.
Requirement fit	**Low**	**Medium**	**High**	**High**
Selected	No	No	No	**Yes**

Figure 6 Collaboration and concurrency control morphological chart

4.3.5 Edit recovery and change safety

The edit recovery and change safety parameter concerns how the tool supports recovery from mistakes and safe exploration of alternatives. This was important because the intended use case involves frequent iteration and restructuring during early-stage design. Operation-based undo and redo, combined with automatic backup, was selected because it supports both fine-grained recovery of recent actions and broader protection against data loss. Simpler approaches such as periodic backups or snapshot rollback alone were considered insufficient for day-to-day concept development.

	Option 1	Option 2	Option 3	Option 4
Name	**Manual saves only**	**Periodic backups only**	**Snapshot rollback only**	**Operation-based undo/redo plus automatic backup**
Picture				
Description	Users rely on manual save points and informal recovery habits.	Periodic backups exist, but there is no fine-grained undo.	Users can revert to saved snapshots, but not undo individual recent actions well.	User actions are recorded so changes can be undone and redone, with backups protecting against larger failures.
Advantages	Minimal implementation overhead.	Protects against catastrophic data loss.	Simpler than a full undo/redo system.	Best support for safe experimentation during early-stage iteration.
Disadvantages	Very weak support for exploratory work and error recovery.	Does not support day-to-day iterative design well.	Too coarse for rapid concept iteration.	More implementation complexity than manual saves or backups alone.
Requirement fit	**Low**	**Medium**	**Medium**	**High**
Selected	No	No	No	**Yes**

Figure 7 Edit recovery and change safety morphological chart

4.3.6 Version and variant management

The version and variant management parameter concerns how different development states and alternative concepts are represented over time. It was especially important because the current process suffers from manual snapshots, variant proliferation, and poor traceability. A branch-based model with editable working states and immutable baselines was selected, since it supports parallel alternatives and controlled maturation much better than linear histories or copied files. This approach requires clearer rules for version handling, but it offers the most defensible solution to the identified workflow problems.

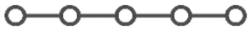
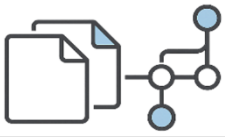
	Option 1	Option 2	Option 3	Option 4
Name	**Manual file copies per variant**	**Linear snapshot history**	**Branch-based variants with editable working state and immutable baselines**	**External file version control on exported data**
Picture				
Description	Each alternative is stored as a separate copied file.	Versions are saved in a linear sequence without explicit branching.	Alternatives are handled as branches, while baselines remain immutable once saved.	Variants are managed through version control on exported files outside the main tool.
Advantages	Easy to understand initially.	Cleaner than raw file copying; provides a basic historical record.	Best support for conditional variants, reversibility, and structured maturation over time.	Strong traceability if users are comfortable with technical workflows.
Disadvantages	Creates bloat, confusion, and weak traceability.	Poor support for parallel alternatives and variant comparison.	Requires explicit rules for baselines, working states, and branch creation.	Pushes core variant management outside the main tool and raises adoption barriers.
Requirement fit	**Low**	**Medium**	**High**	**Medium**
Selected	No	No	**Yes**	No

Figure 8 Version and variant management morphological chart

4.3.7 Navigation and scalability for large systems

The navigation and scalability parameter concerns how the user can explore and manage the system as the system grows in size and complexity. This was a key consideration because one of the main identified problems was that large systems become difficult to overview and work with. A filterable multi-view workspace with search, tables, graph views, and clustering support was selected, since it provides several complementary ways of navigating the same data. While this requires more interface design effort than a single all-in-one view, it is the most suitable approach for scalable use.

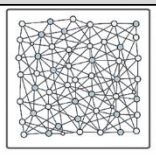
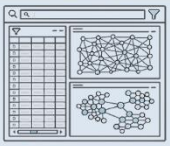
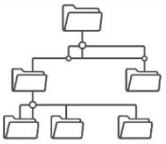
	Option 1	Option 2	Option 3	Option 4
Name	**Single all-in-one sheet**	**Single all-in-one graph canvas**	**Filterable multi-view workspace with search, tables, graph, and clustering**	**Hierarchical subsystem tree only**
Picture				
Description	The full system is managed in one large sheet.	The full system is managed in one large graph view.	Users move between synchronized views and filter the system to relevant scopes.	Users mainly navigate through hierarchical subsystem breakdowns.
Advantages	Simple mental model; easy to export as one table.	Strong visual system overview at small to medium scale.	Best support for scale, navigation, and different user tasks.	Useful for decomposition and top-down browsing.
Disadvantages	Breaks down as system size grows.	Also becomes cluttered as system size grows.	More interface design effort is required.	Weak for cross-cutting relations that do not follow the hierarchy cleanly.
Requirement fit	**Low**	**Medium**	**High**	**Medium**
Selected	No	No	**Yes**	No

Figure 9 Navigation and scalability for large systems morphological chart

4.3.8 Extensibility and schema evolution

The extensibility and schema evolution parameter concerns how the tool can adapt to future information needs without losing structure. This was important because the requirements include support for new attributes, status checklists, and future expansion of the tool's scope. Controlled custom attributes and checklists on top of a stable core schema were selected, since this approach provides flexibility while preserving consistency and filterability. In contrast, unstructured extensions were considered too weak from a data-quality and traceability perspective.

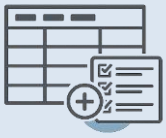
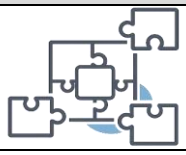
	Option 1	Option 2	Option 3	Option 4
Name	**Fixed schema only**	**Unstructured notes and free-text extensions**	**Controlled custom attributes and checklists on top of a stable core schema**	**Fully plugin-defined domain model**
Picture				
Description	All attributes are predefined and cannot be extended by users.	New information is stored informally in notes or arbitrary text fields.	A stable core model exists, but users can add controlled extra fields when needed.	Entire new domain entities and behaviors are added through plugin mechanisms.
Advantages	Strong consistency and simple validation.	Very flexible in the short term.	Balances consistency with adaptability; supports future checklists and status fields.	Maximum long-term extensibility.
Disadvantages	Weak adaptability to changing user needs.	Weak structure, weak filterability, and weak single source of truth.	Requires design rules for how extra fields are defined and used.	Too complex for the scope and maturity level of the thesis tool.
Requirement fit	**Medium**	**Low**	**High**	**Medium**
Selected	No	No	**Yes**	No

Figure 10 Extensibility and schema evolution morphological chart

4.3.9 External editing and analysis exchange

The external editing and analysis exchange parameter concerns how data from the tool can be transferred to other environments for review, analysis, or further processing. This was important because export and structured bundling are explicit requirements, while future development may also require other software tools to interface with the same data and support later design stages. For this reason, two compatible options were selected. CSV and Excel import/export provide a practical and widely accessible exchange method for current engineering workflows, while direct API exchange provides a path for future software integration and further use of the data in downstream tools. More specialized CAD or PLM integration was not selected because it extends beyond the primary scope of the present work.

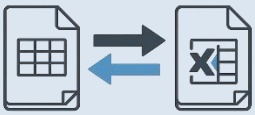
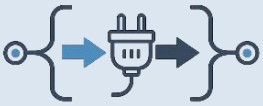
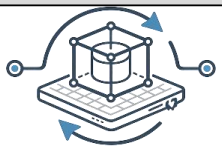
	Option 1	Option 2	Option 3	Option 4
Name	**Manual copy/paste exchange**	**CSV and Excel import/export bundles**	**Direct API exchange**	**CAD or PLM integration**
Picture				
Description	Users transfer data manually between tools.	Structured bundles are exchanged through widely used tabular file formats.	External use depends mainly on software-to-software interfaces.	Data exchange is centered on engineering platform integration.
Advantages	Minimal implementation effort.	Strong compatibility with common engineering workflows and low adoption threshold.	Strong automation potential in more mature tool ecosystems.	Can support later lifecycle integration.
Disadvantages	Error-prone and weakly traceable.	File import/export requires mapping and consistency checks.	Too dependent on other software ecosystems and less useful for ad hoc analysis.	Too far from the core early-stage need in this thesis.
Requirement fit	**Low**	**Medium**	**High**	**Low**
Selected	No	**Yes**	**Yes**	No

Figure 11 External editing and analysis exchange morphological chart

4.3.10 Access and platform strategy

The access and platform strategy parameter concerns how users access the tool in practice and how the main system is made available across platforms. This was important because the tool is required to run across operating systems while also supporting collaborative work on shared data. A browser-based client-server solution was selected as the primary user-facing access mode, since it provides strong cross-platform accessibility together with centralized data management and collaboration support. The concept does not exclude future software-to-software access to the same underlying data, as this is addressed separately through the selected API-based exchange direction for later tool integration and downstream design support.

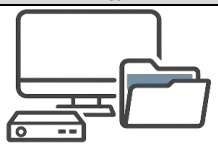
	Option 1	Option 2	Option 3	Option 4
Name	**Local desktop application with local files**	**Browser-based client-server application**	**Hybrid desktop client with central backend**	**Command-line or scripting-first tool**
Picture				
Description	The tool runs locally and stores data on the local machine or shared folders.	Users access a central service through a web browser.	A packaged client exists, but core data and collaboration remain centralized.	Users interact mainly through scripts or technical command-line workflows.
Advantages	Simple local deployment; familiar desktop-tool feel.	Strong cross-platform support and low client installation burden.	Can combine richer local behavior with central collaboration.	Strong for automation by expert users.
Disadvantages	Harder to support robust collaboration across users and sites.	Requires server infrastructure and network access.	More deployment complexity than a pure browser solution.	Too high a barrier for intended users without programming background.
Requirement fit	**Medium**	**High**	**High**	**Low**
Selected	No	**Yes**	No	No

Figure 12 Access and platform strategy morphological chart

4.4 Identified design workflow

In addition to the five workflow problems identified in Section 4.1, the empirical observations and discussions also made it possible to define a high-level design workflow for early-stage electrical system design work. This workflow is shown in Figure 13 and represents the intended progression of work from early concept reasoning toward increasingly structured system design data and implementation-related outputs.

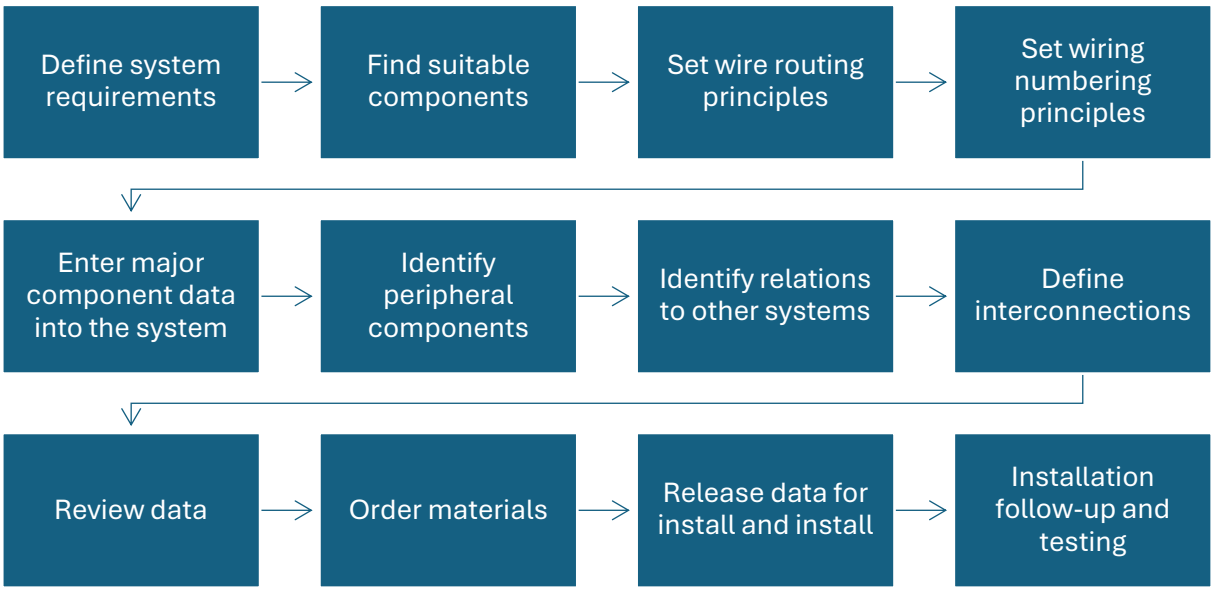


Figure 13 High-level design workflow for early-stage electrical system design work

The workflow was useful because it clarified which stages the ESDT needed to support, from early system definition to more detailed, implementation-relevant design data. It should not be read as one strictly linear pass through the whole electrical system. After the overall principles have been set, the steps from entering major component data onward are repeated as additional subsystems are added, built out, reviewed, and tested. The workflow should therefore be understood as a high-level iterative process, consistent with the build-measure-learn way of working in the studied context. It does not imply that the current tool environment supports this process well. Rather, it describes an intended progression that is in practice affected by several recurring issues in the existing tool environment.

One example concerns connector definition and part-number handling. A connector may be used in several locations, but the associated information must be entered repeatedly for each occurrence. If the part later becomes obsolete or is replaced, each occurrence must then be updated manually. This illustrates how the intended workflow can be followed, but only with repeated manual work and a continuing risk of inconsistency.

4.5 ESDT Prototype description

This section describes the ESDT in terms of its software structure, persisted data model, available user-facing views, and implemented mechanisms for collaboration and version history. Prototype delimitations are stated explicitly where relevant.

4.5.1 ESDT overview

The ESDT is implemented as a browser-based client coupled to a separate Go backend and a relational database. As shown in Figure 14, the browser frontend contains the application flow,

the global login, the Work Tree, the editor, and a separate admin page, while the Go backend provides routing, service logic, and access to the persisted data. HTTP endpoints are used for ordinary data and workflow operations, and a WebSocket channel is used for live collaboration state. In practical use, the backend acts as the central authority for persisted system design data, branch and version handling, import and export, undo and redo, user and permission management, and edit coordination.

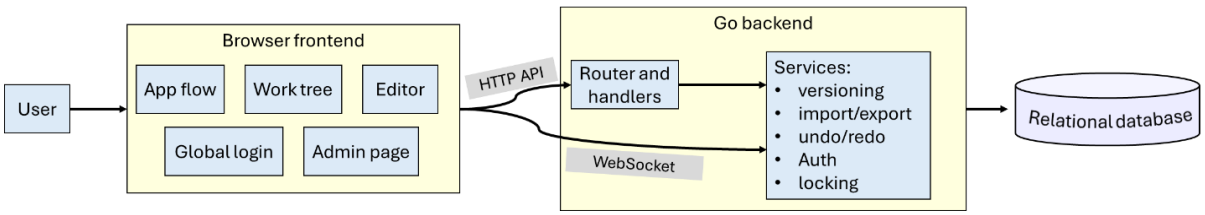


Figure 14 System-level overview of the ESDT

Beyond this logical split, the ESDT is packaged as a containerised runtime. The repository includes Dockerfiles for the frontend and backend together with a Docker Compose stack that starts the frontend, backend, PostgreSQL, Redis, and MinIO services. Plain Kubernetes manifests are also provided for namespace, configuration, secrets, PostgreSQL, Redis, MinIO, database initialisation, backend, and frontend deployment. The overall setup therefore supports both a local container-based runtime and a more structured orchestration path.

The early prototype used SQLite for simplicity, while the later implementation moved toward PostgreSQL in order to use a more capable backend and support a more scalable runtime arrangement. Because the system design is stored in a relational SQL-based form, it should be relatively easy to migrate the data to other database formats or standards.

4.5.2 Data model overview

The ESDT uses a relational data model that is organised into designs, branches, and versions. The description here focuses on the implemented entities and relations rather than on intended future extensions. As shown in Figure 15, the version level is the scope at which the main system entities are stored. Each branch contains one working version for active editing, while saved versions are retained as read-only snapshots. Version state is therefore represented explicitly in the persisted model rather than being handled only as frontend state.

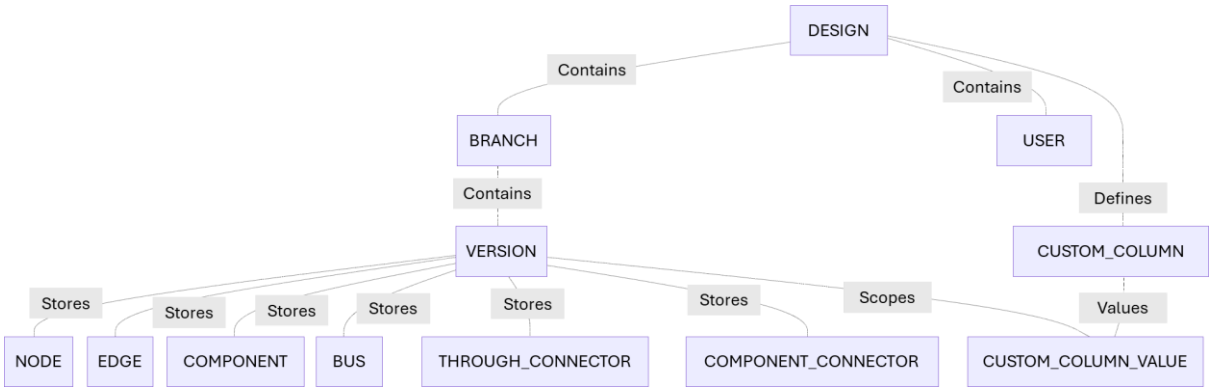


Figure 15 Data model and version scoping used by the ESDT

The system itself is represented through tables for nodes, edges, components, buses, through connectors, and component connectors. In addition, the persisted model includes custom column definitions and values, user records, lock records, operation history, datasheet links, and user view preferences. Each relevant entity row carries explicit design_id, branch_id, and

version_id values, and backend services use these values to scope reads and writes to the intended version state.

Identity is not handled through one universal key across all table types. Instead, the implementation uses table-specific identifiers, such as a node reference for nodes and an edge hash identifier for edges. The graph view is built from the same persisted rows used in the tabular sheets, with fields such as node position and edge routing stored as data attributes in the relational model. Standard columns and user-defined columns are merged through a shared schema registry, which provides one structural basis for editing, filtering, export, and graphical presentation.

4.5.3 User views and workflows

The user-facing workflow begins at the global login view shown in Figure 16. From this page, existing users can sign in and new users can be created, but the self-service creation path is limited to read-only accounts. Edit permission must therefore be assigned later through the admin interface. The same entry view also provides access to the admin page, which places authentication and basic account setup before the design workflow itself.

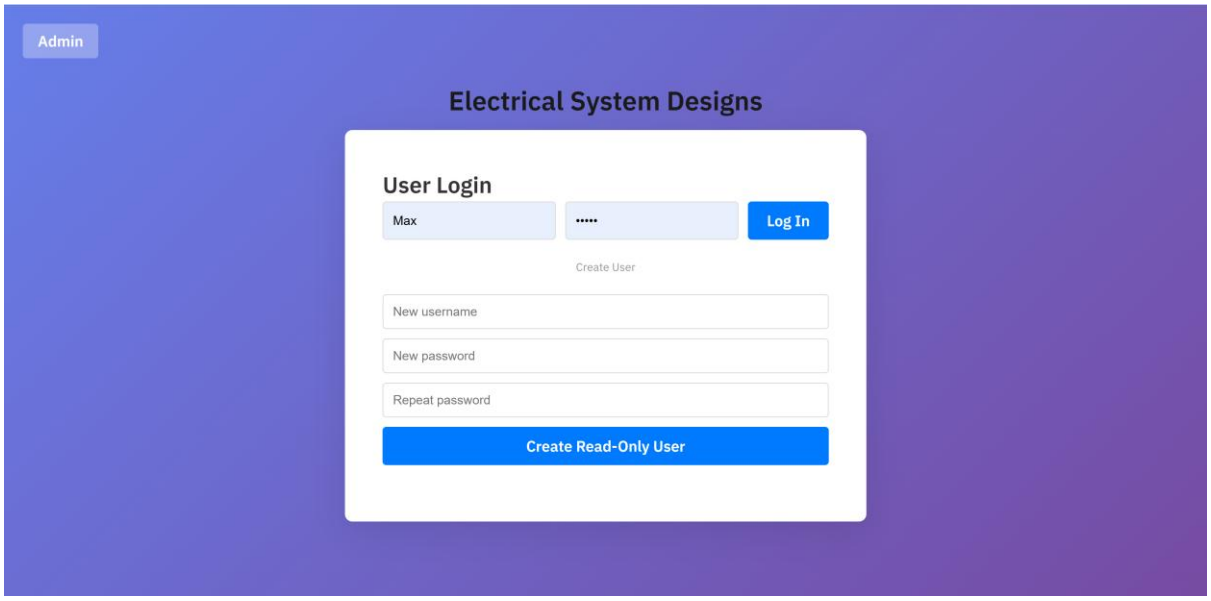


Figure 16 Global login view with entry to the admin page

The admin page is shown in Figure 17. This view is used for role assignment, activation and deactivation, password reset, deletion where permitted, and testing-mode control. In practice, it provides the mechanism by which read-only users can be granted editor permissions and therefore forms part of the ordinary workflow around the collaborative environment rather than a separate maintenance function.

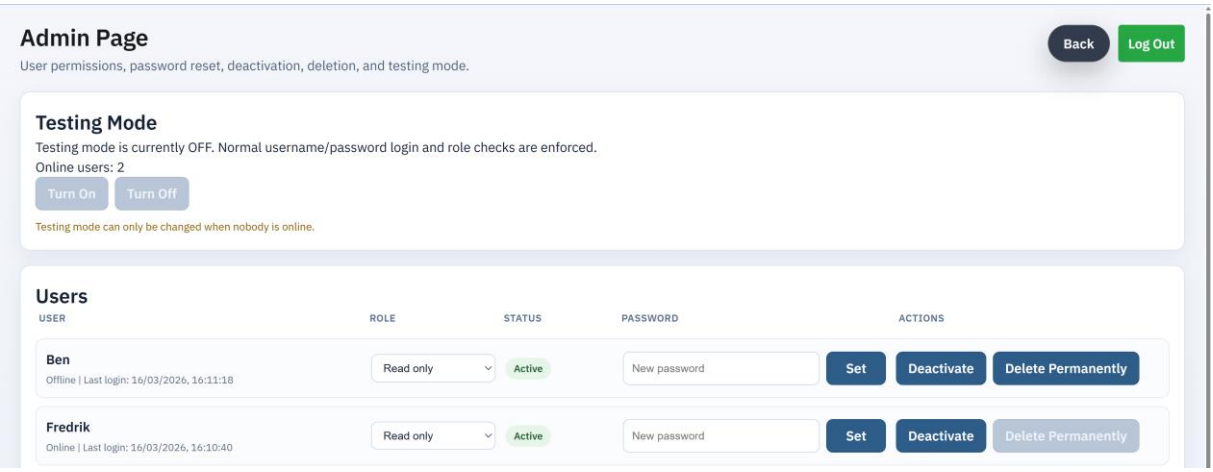


Figure 17 Admin page for user and permission management

After login, the normal design workflow continues to the Work Tree. Figure 18 summarises the version concepts that structure this part of the workflow. A design can begin either as a new tree or through an import operation, and this creates a working version that represents the current editable state of a branch. Saving creates a read-only saved version, while freezing turns the current working version into a read-only branch endpoint. If further development is required from a saved version or a frozen working version, a new branch can be created from that state, which produces an independent editable copy. The ESDT does not include a dedicated branch merge function, so reuse across branches is instead handled manually through the existing copy-and-paste workflow in the editor.

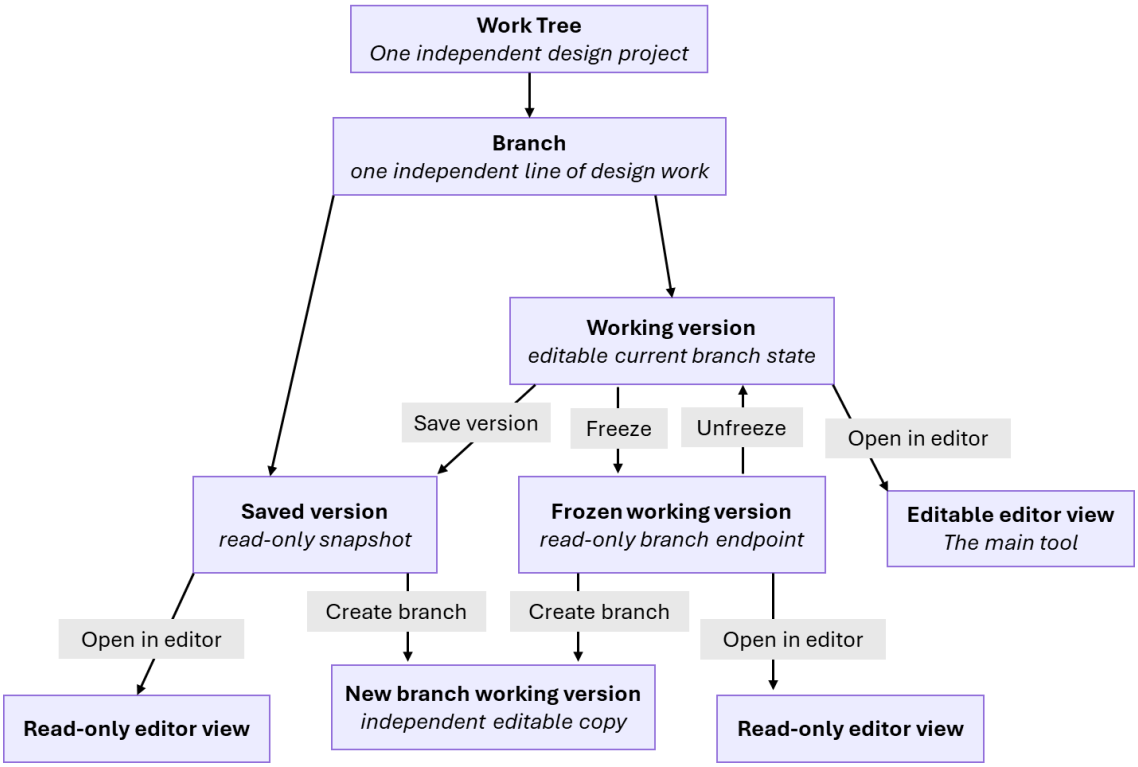


Figure 18 Work Tree concepts and available actions

Figure 19 shows the corresponding work tree view in the implemented interface. The work tree is presented as a graph-based version history rather than as a simple list, which supports navigation across branches and version states. From this view, the user can create a new tree, import data

into a new branch, save versions, freeze and unfreeze working versions, and open either editable or read-only editor states. Import preview and commit are part of the same flow, which means that a new design line can begin either from scratch or from external input.

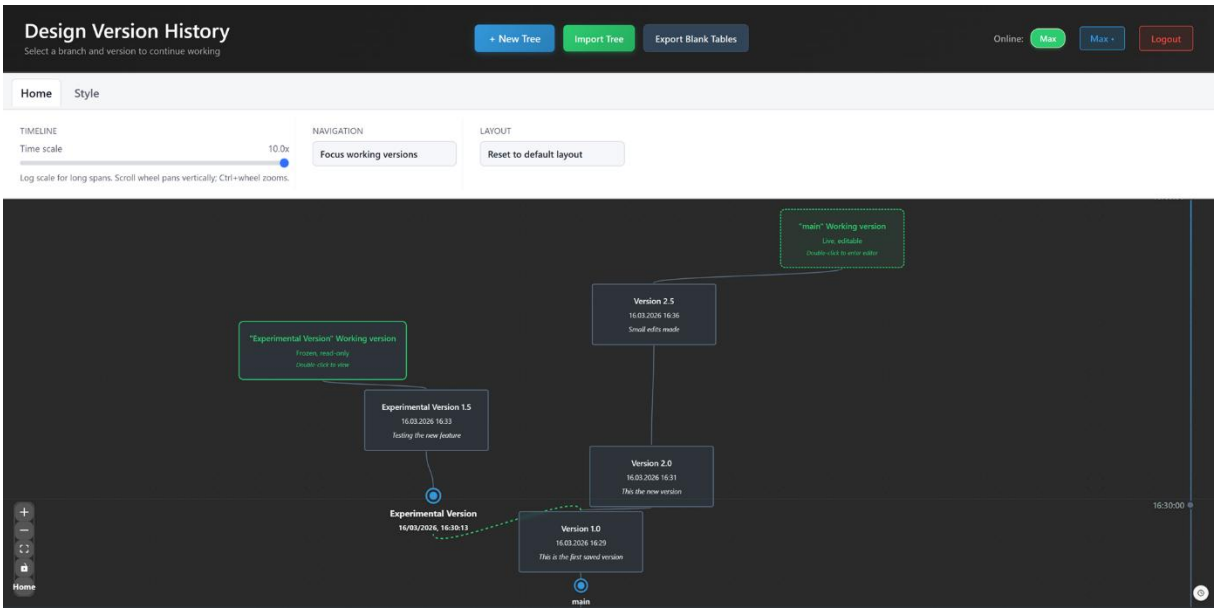


Figure 19 Work Tree view showing the work history

The next stage of the workflow is the editor. The main editor is a Dockview-based workspace with coordinated panels for graphs and tabular sheets. As shown in Figure 20, graphical structure and structured sheet data can be inspected within the same editor session. The graph panel and the sheets do not represent separate copies of the system. Instead, they operate on the same underlying branch and version state, which allows the user to move between visual and tabular views without changing context. In practical use, the graphical view is intended as the primary interaction mode for early-stage system design work, because it supports faster restructuring and a more intuitive overview of connected systems than a purely tabular presentation.

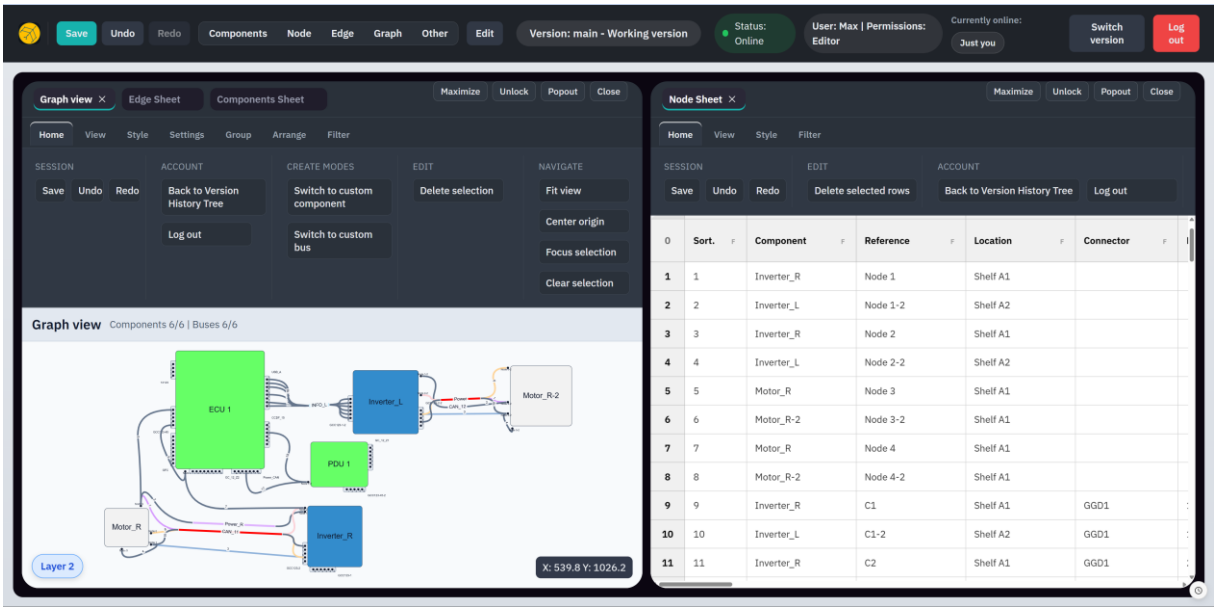


Figure 20 Editor workspace with the graph view and a node sheet open at the same time

The graph panel should be used as the main stand-alone working view for the ESDT. Figure 21 shows this intended graph-first mode without the adjacent table, which makes the topology, placement, and connections of the system directly visible during early-stage construction and restructuring. The ESDT is therefore oriented toward day-to-day work in the graphical view rather than toward continued dependence on the tabular sheets. The tabular views are retained mainly as a transition aid for users coming from spreadsheet-based work and as a fallback when a required action is available in the sheet interface but has not yet been given an equally practical graph-based interaction. For larger systems, manageability is supported through a combination of work tree navigation, graph interaction, data grouping, and filtering.

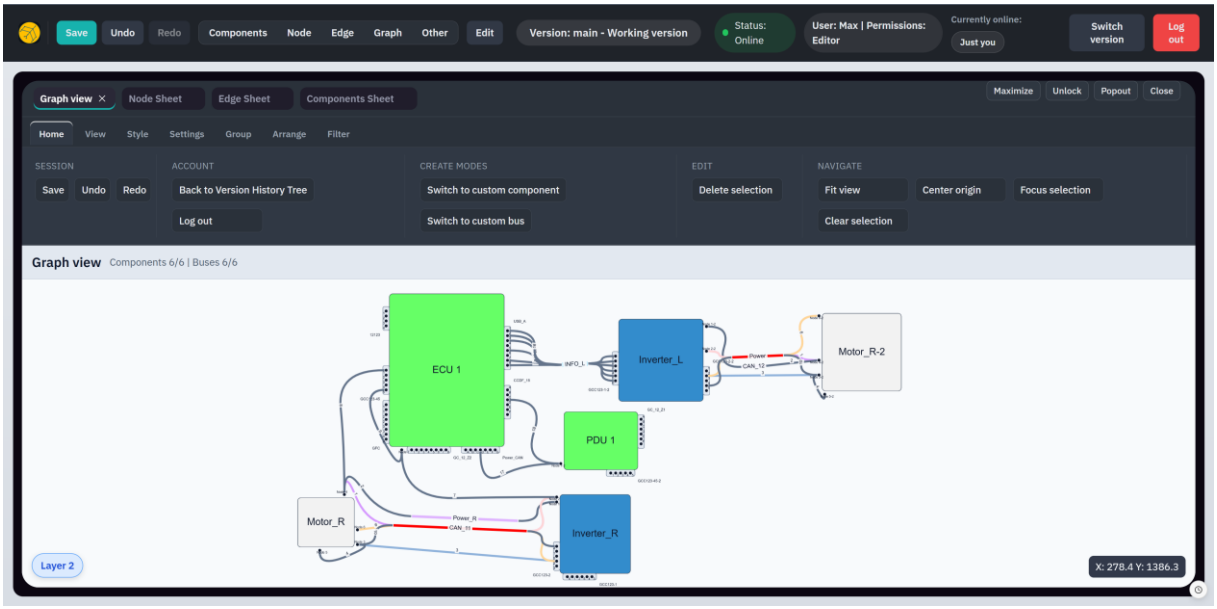


Figure 21 Graph view showing an example electrical system design

Graph-based editing is supported through direct mouse interaction together with lightweight keyboard support. As shown in Figure 22, users can rearrange the system by dragging nodes and components, expand selections through Ctrl or Cmd assisted interaction, and use graph navigation commands such as fit view, focus selection, centre origin, and clear selection. Creation modes in the graph ribbon allow new nodes or components to be placed directly and new edges or buses to be created by dragging between endpoints. For structural tasks such as arrangement, connection definition, and rapid restructuring of alternatives, this provides a considerably more direct working mode than entering relationships and positions one value at a time in spreadsheet cells.

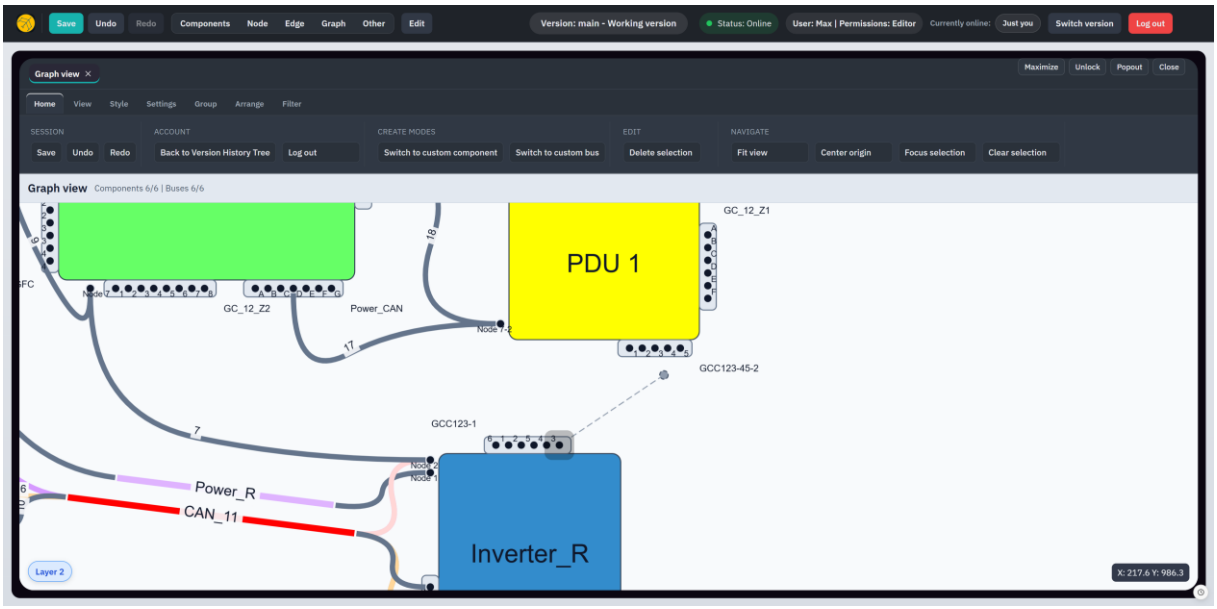


Figure 22 Graph-based connection creation through direct drag interaction

The graph view also combines direct interaction with modal editing when more detailed input is required. Quick create modes allow immediate placement, while custom create modes open the corresponding creation modal so that attributes can be entered at creation time. Existing nodes, edges, components, and buses can likewise be opened in edit modals directly from the graph, for example through double-click or context-menu actions. These modals are not fixed in one rigid form. Their layout can be adapted between different presentation modes, and the visible fields can be reduced, restored, and arranged to suit the user's preferred input flow. Because these choices are stored as user-specific preferences, the modal workspace can be tuned toward either denser technical input or a more reduced and guided editing view. In this way, structural work remains graph-centred while detailed attribute entry is still available without leaving the graphical context.

Selection-based editing is extended through the multi-edit modal shown in Figure 23. When several nodes, edges, components, or buses are selected, one modal can be used to apply the same property changes across the whole selection. This supports rapid repeated edits in the graph workflow and reduces the need to repeat the same change row by row in a sheet.

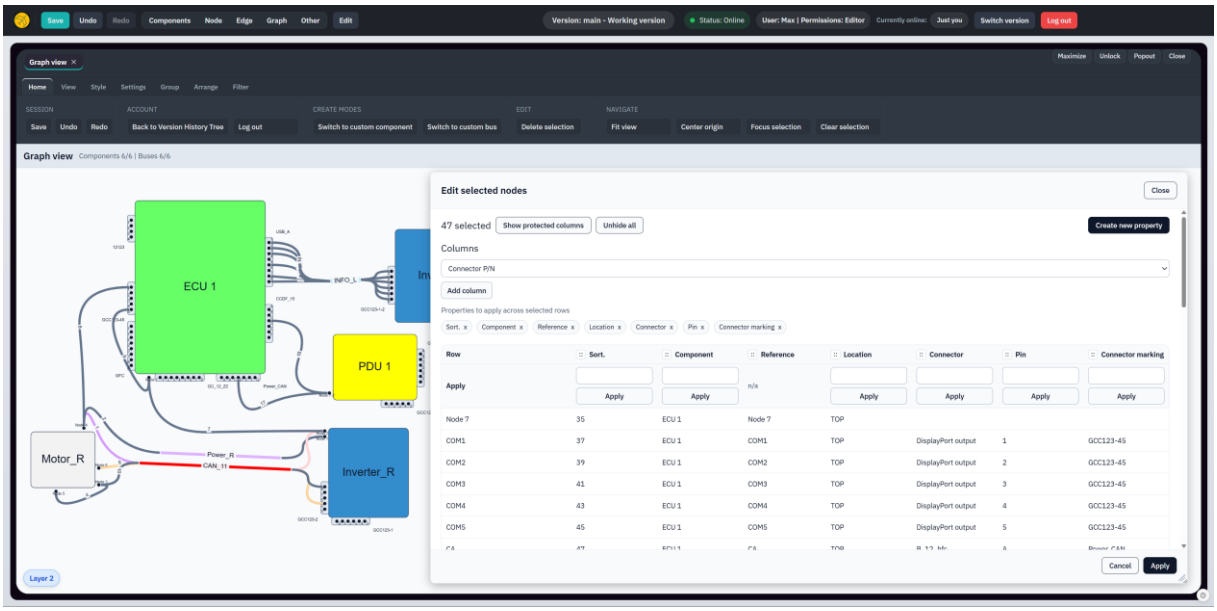


Figure 23 Multi-edit modal used to apply shared property changes across a graph selection

Sheet-based editing nevertheless remains fully available for structured attribute work. Figure 24 shows a table-focused editor view in which entity attributes are edited in rows and columns. In the present prototype, this table layer mainly serves two purposes. First, it provides a familiar entry point for users transitioning from the earlier spreadsheet-based environment. Second, it provides a practical fallback for operations that still remain more complete or efficient in tabular form while corresponding graph-based interactions are still limited or not yet implemented. The sheet interface therefore preserves continuity with earlier working practices, while the intended day-to-day use of the ESDT remains centred on the graphical interface. The table views add integrated filtering, custom columns, datasheet linking, undo and redo, live lock handling, and direct connection to the shared versioned data model. The ESDT also preserves spreadsheet compatibility through CSV and XLSX export together with corresponding import flows for the main entity tables, so system data can be taken out to Excel-compatible form and later brought back into a new working branch when needed. Procurement-related information appears mainly as attributes and export fields within these sheets rather than as a separate procurement workspace.

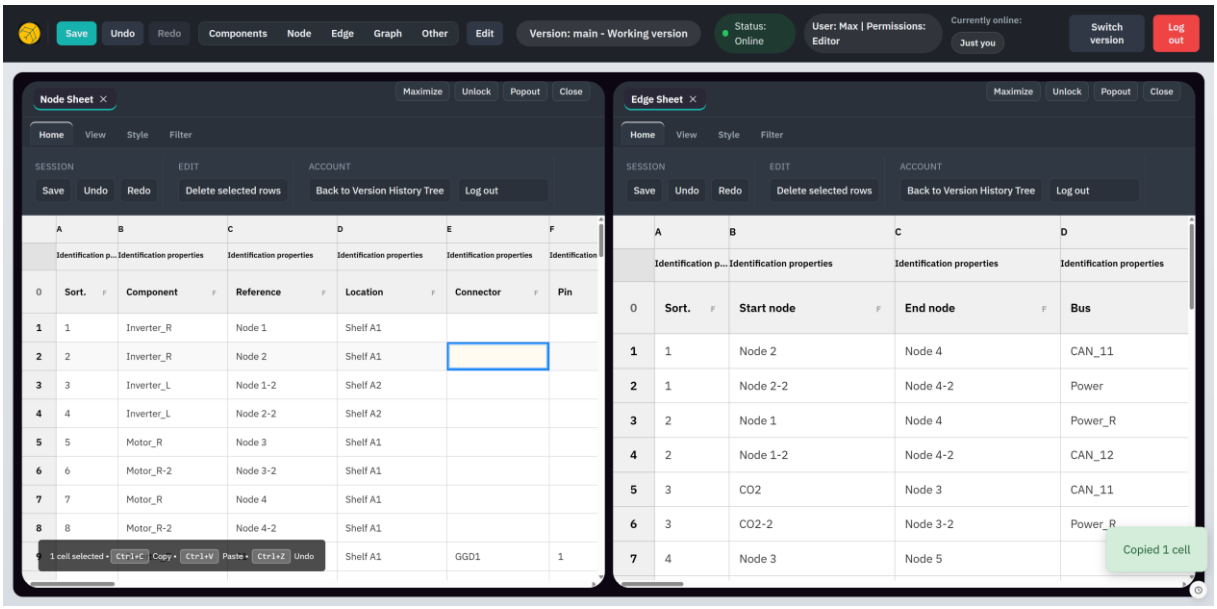


Figure 24 Table-focused editor view

4.5.4 Collaboration and version history implementation

The collaboration model combines live presence information, explicit locks, role-based write permission, and versioned working states. As shown in Figure 25, editing of a cell begins with lock acquisition, after which the change is persisted through the backend and propagated to other clients. If another user tries to edit the same cell while the lock is already visible in the interface, the frontend blocks that action before editing begins. The collaboration model therefore supports simultaneous access to the same system design, but it does so through controlled locking rather than through unrestricted concurrent editing.

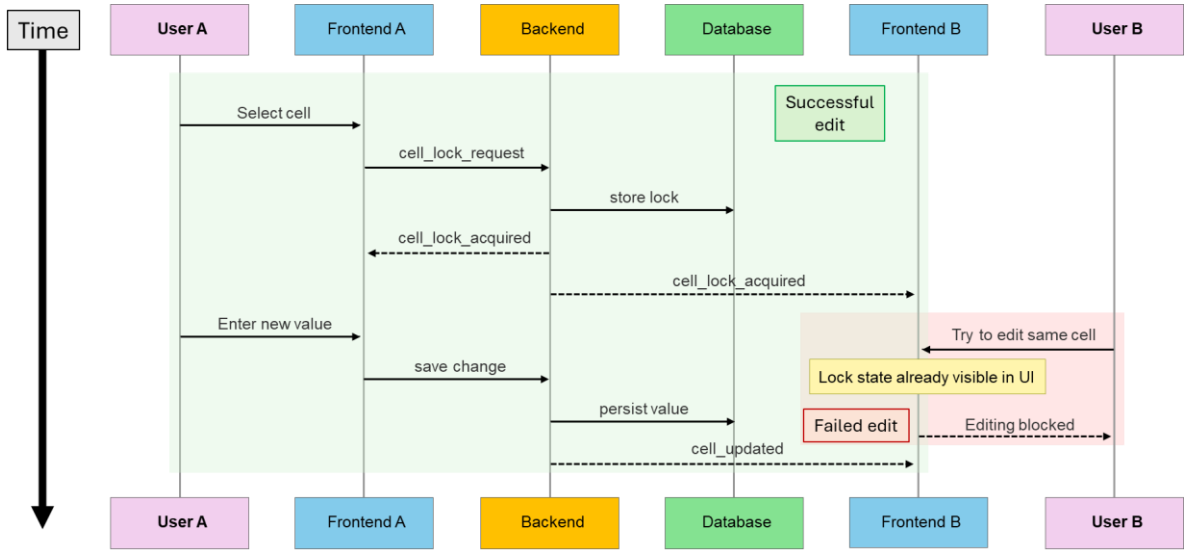


Figure 25 Two-user edit-control flow

The same mechanism is visible in the editor context in Figure 26. The screenshot shows the active sheet together with the wider editor state, which makes the cell-level scope of the coordination visible within the normal workspace rather than as a separate modal process. The selected cell is marked directly in the table, and another user is indicated on that cell.

0	Sort.	Component	Reference	Location	Connector
1	1	Inverter_R	Node 1	Shelf A1 Ben	
2	2	Inverter_L	Node 1-2	Shelf A2	
3	3	Inverter_R	Node 2	Shelf A1	

Figure 26 Cell-level edit block in the tabular view

Edit coordination is therefore implemented at cell level rather than at whole-table or whole-branch level. The backend stores cell locks persistently, the frontend requests individual or bulk locks before editing, and locks are released explicitly or expire after a defined interval. The backend also distinguishes editor and read-only users, and mutating routes require editor permission unless testing mode is active. Protected fields require a reason before editing is permitted, and these edits are recorded separately. The result is an explicit edit-control model that preserves control over the design state without introducing automatic merge behaviour during concurrent work.

4.6 Demonstration case and evaluation outputs

This section presents the demonstration case used to evaluate the developed prototype ESDT and assess requirement coverage. The case is based on the creation of a representative electric propulsion architecture in the tool and was selected because it matches the application area of the thesis while remaining limited enough to present clearly. The goal of the case is not to produce a final aircraft design or a fully optimized propulsion system, but to show how the ESDT prototype supports the main steps of early-stage electrical system design work and how the resulting system data can be structured, edited, reviewed, and reused.

The evaluation follows the logic defined earlier in the method chapter and the design workflow identified in Section 4.4. The case therefore moves from initial system design definition towards a more structured and connected system representation and is then used to assess requirement coverage. Functions that were already described in detail in Section 4.5 are not repeated here in depth. Instead, this section uses the demonstration case to show how those functions are applied in a coherent workflow and refers back to the earlier prototype description where relevant.

4.6.1 Electric powertrain validation case

The selected validation case is a simplified battery-electric propulsion architecture for a fixed-wing UAV and should be understood as a representative UAV-scale turbine-generator replacement case, consistent with the application framing introduced earlier in the thesis. A render of an older concept of the chosen UAV case can be seen in Figure 1. The system was defined as a single-motor, single-propeller propulsion chain with two battery packs and supporting systems. The system includes a high-voltage DC main power path, low-voltage auxiliary supply, inverter-fed motor drive, and representative sensing, protection, and control functions. This makes the case large enough to exercise the intended tool functions while still remaining manageable for presentation.

The main system design includes two battery packs, battery management, maintenance cut-off functions, reverse current protection, a main contactor, pre-charge circuit, high-voltage junction and distribution elements, inverter, electric motor, propulsion control unit, avionics or flight

computer interface, a low-voltage battery, DC/DC converter, and a representative set of voltage, current, temperature, and speed sensing functions. In this way, the case includes both the main propulsion chain and the supporting functions needed to test structuring, interconnection handling, and navigation across a more realistic system.

The validation case was informed by a real internal use context. Several architectural choices, component definitions, and interface structures were based on real component information, real datasheets, and actual workflow needs. However, some details were generalized, modified, or partially obfuscated afterwards to avoid disclosure of sensitive system information. The case should therefore be understood as representative rather than as a direct reproduction of the original internal system.

4.6.2 Development of the demonstration system

The first stage was to define the main propulsion architecture and add the major system components. At this point, the purpose was to establish the main system structure and the main physical and functional blocks before more detailed interface data were entered. The initial graph therefore focused on the battery packs, inverter, electric motor, battery management system, propulsion control unit, and avionics or flight interface as shown in Figure 27.

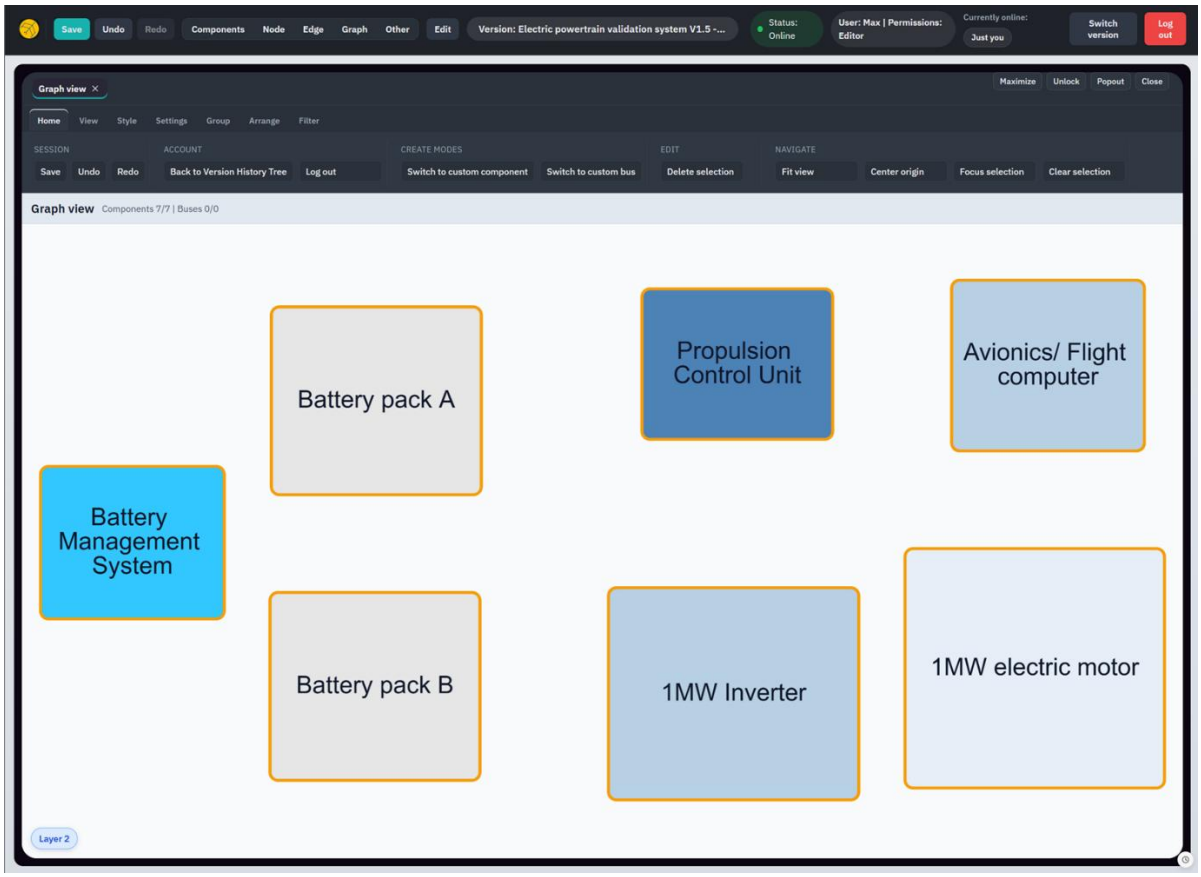


Figure 27 Major propulsion architecture components added

In the second stage, the major components were enriched with more detailed interface and connector information. Representative connector and pin definitions were entered based on real or real-inspired datasheet information, so that the system moved from a rough block-level structure towards a more explicit architecture representation as shown in Figure 28. This stage is important because it shows that the ESDT does not only support block placement, but also

supports structured entry of component-level interface data that later interconnections can build on.

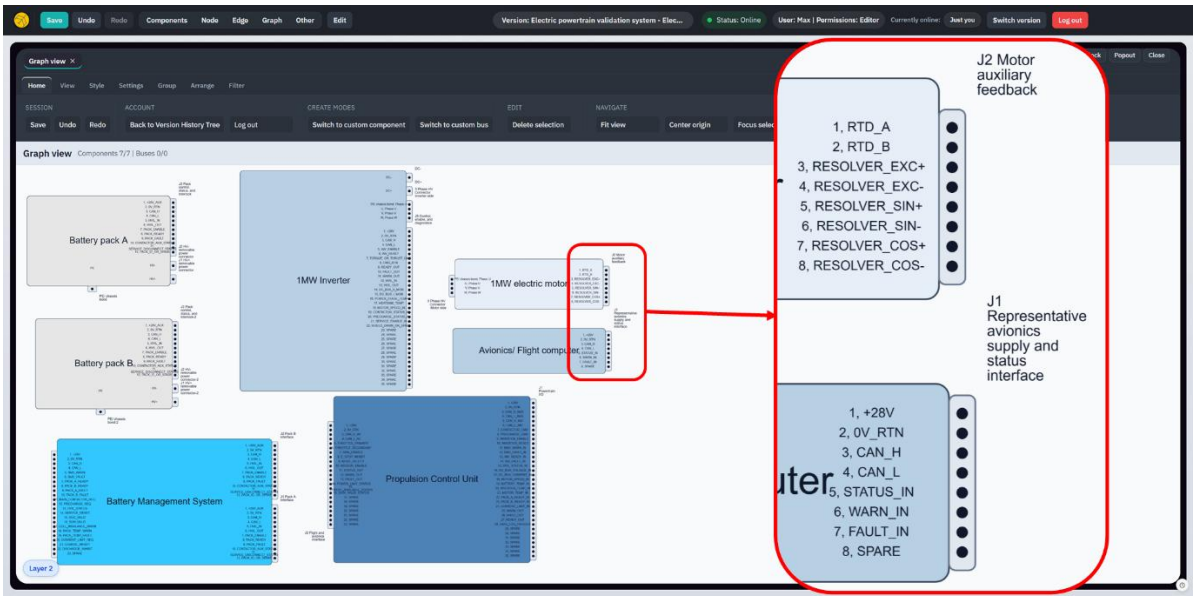


Figure 28 Major components after interface and connector definition

In the third stage, the system design was extended with auxiliary and supporting elements. This included supporting power supply elements, sensing functions, junction and distribution elements, and other peripheral components required to make the system more representative of a real design situation. This stage of the process is seen in Figure 29. At this point the system began to move beyond the main propulsion chain and towards a more complete electrical system design that also includes control, monitoring, and support functions.

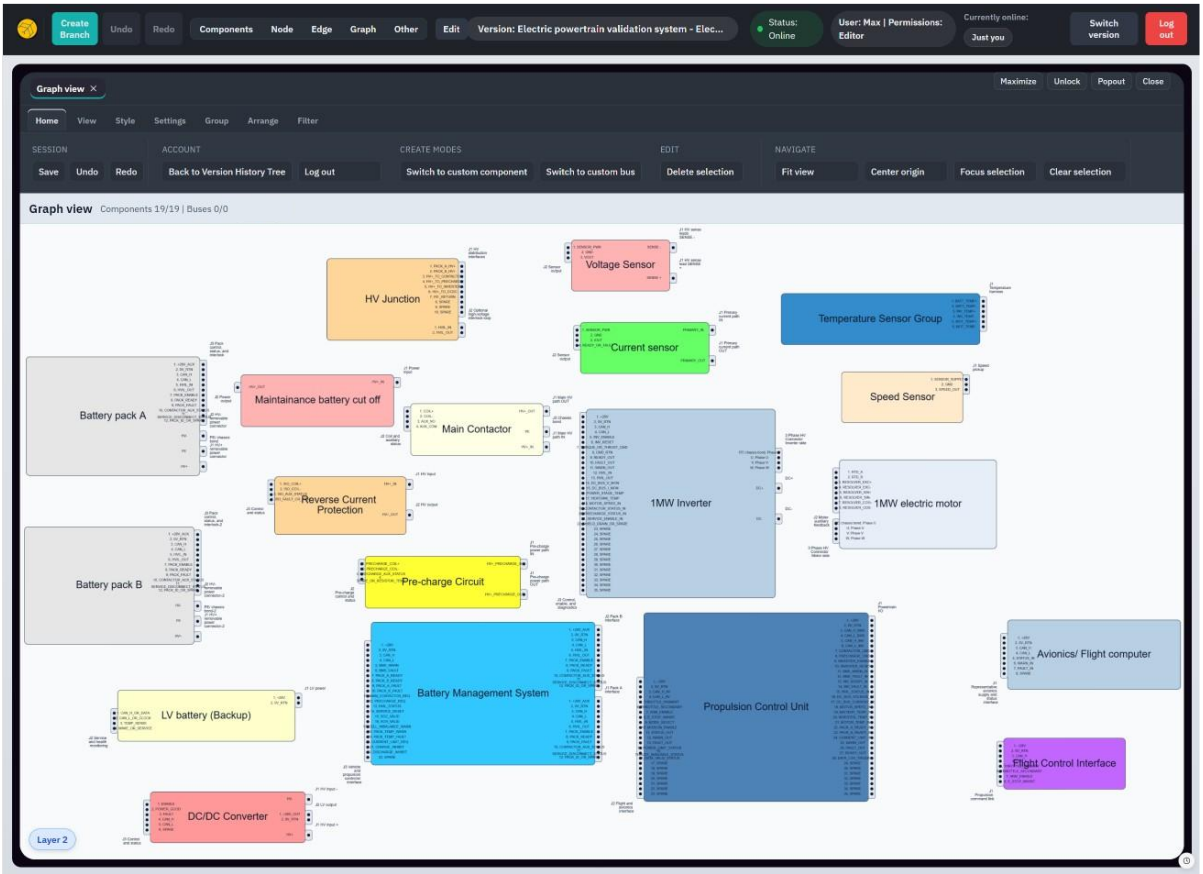


Figure 29 Extended system design with auxiliary and supporting components

In the fourth stage, seen in Figure 30, the system was wired up, arranged, and bundled into a more complete connected representation. This stage included the creation of interconnections between the previously defined component interfaces and the later cleanup of the resulting system view. As expected, the system became significantly denser once all interconnections were present. This reflects a realistic challenge in larger systems and therefore also forms part of the evaluation. At the same time, the final connected system shows that the ESDT is able to maintain a detailed representation in which components, interfaces, wires, and buses remain linked within the same underlying data structure.

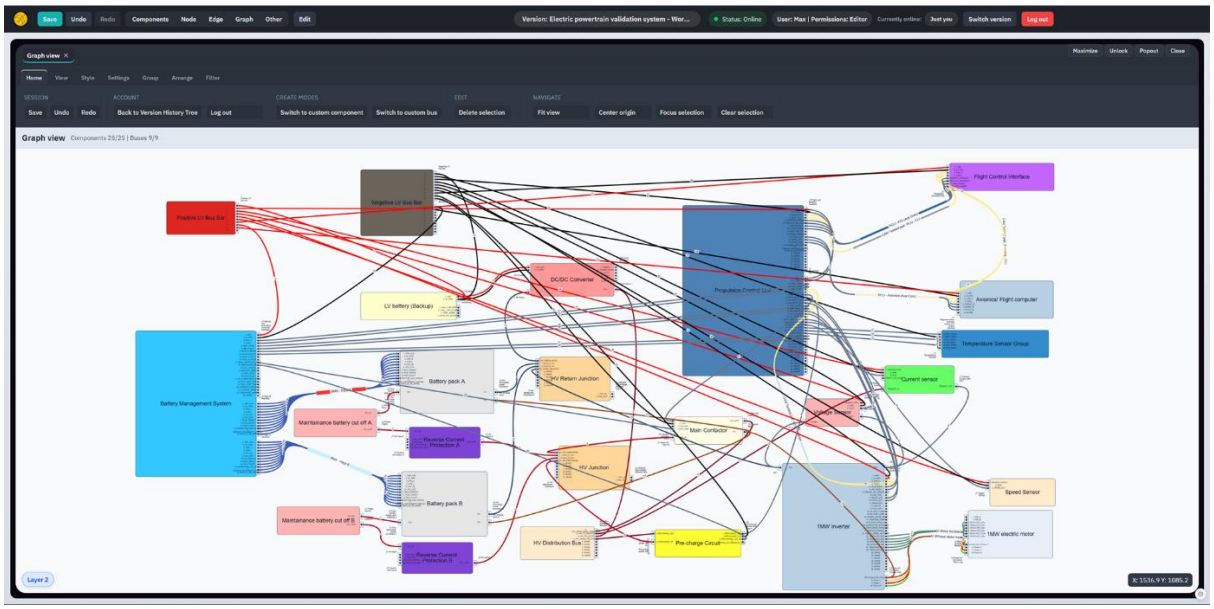


Figure 30 Final wired and bundled electric powertrain validation case

The staged development shown in these figures reflects the intended progression from major system definition to more detailed and structured system data. It also mirrors the practical workflow described earlier in Figure 13, where major components are identified first, interface and interconnection logic are then added, and the system is gradually matured into a more reviewable and implementation-relevant representation.

4.6.3 Additional demonstrated functions and user feedback

Beyond the staged creation of the validation case itself, the same case was used to demonstrate additional functions that are relevant to requirement coverage.

One important example is grouping and filtering. As the complete system becomes denser, it is not always most useful to view the whole system at once. The ESDT therefore allows the user to group components and related objects into categories such as subsystems, and to filter the visible graph accordingly. In the example shown in Figure 31, this is used to isolate selected subsystem content and make local structure easier to understand. This was one of the most important functions for handling larger systems in practice, since it allows the user to move between whole-system overview and subsystem-focused detailed view without having to maintain separate disconnected representations.

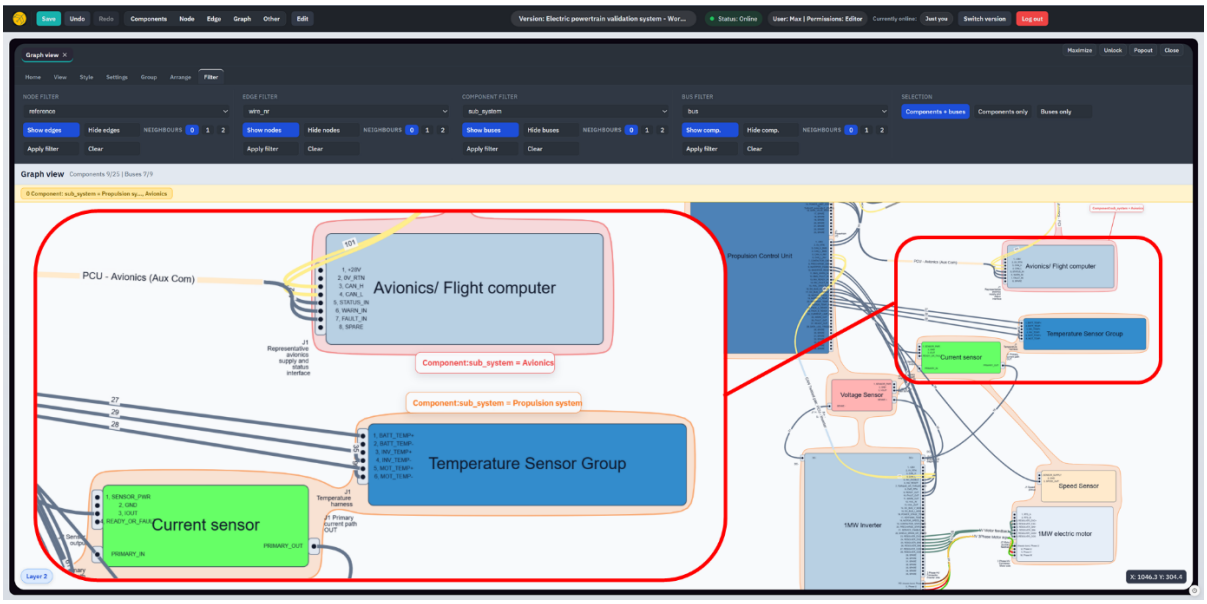


Figure 31 Grouping and filtering of subsystem content

This function was also important beyond the main design task. During prototype interaction, the grouped and filtered graph was perceived as easier to understand than the earlier purely tabular workflow. In particular, it made it easier to trace connections, isolate one subsystem, explain the system to someone else, and see installation-relevant details. This is especially relevant to the requirements related to build understanding, orientation for less initiated users, clustering, and general intuitiveness.

A second important example is the direct link between graph and table views. The ESDT allows the user to move from graphical context to the corresponding structured table data, which helps connect visual understanding with detailed attribute work. In Figure 32, a selected graph element is highlighted in the table, where installation instructions, procurement-relevant information, and custom columns are visible. This supports a workflow in which the graph is used for system understanding and navigation, while the tables remain available for dense attribute editing and export-oriented data handling.

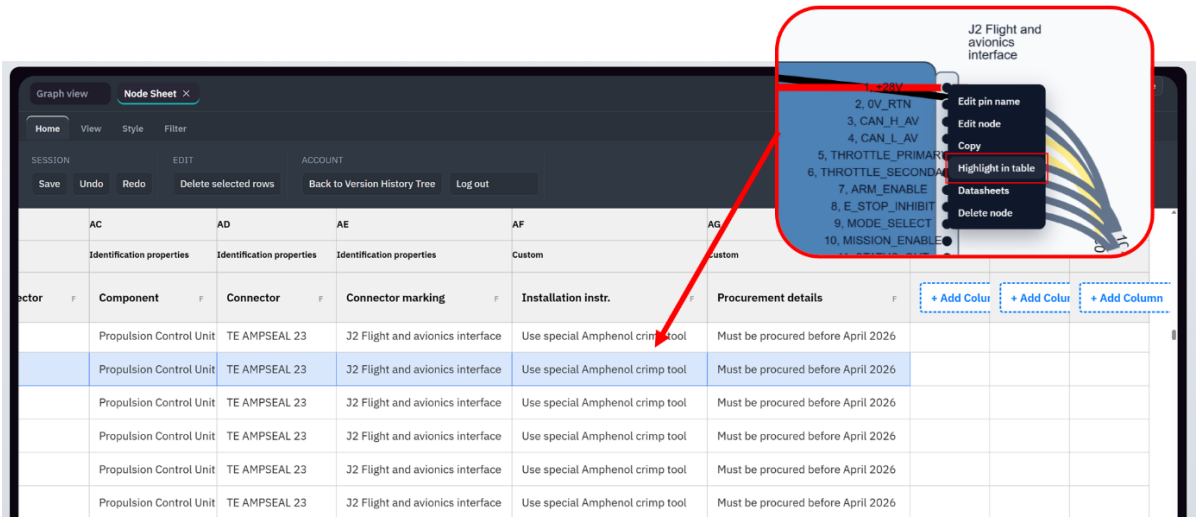


Figure 32 Graph-to-table highlighting with installation and procurement-related fields

This linked workflow supports several requirements at once. It improves procurement-related filtering and list compilation, supports custom attributes and checklist-like extension, and reinforces the single source of truth principle by showing that the graph and table views are different views of the same underlying data rather than separate manually synchronised documents.

The handling of bundled wires and buses was also demonstrated in the validation case, seen in Figure 33. While splice handling in the present prototype is still simpler than a full physical harness modelling workflow, the graphical representation already provides a clearer and more direct way to understand bundles and branch-like structures than the previous spreadsheet-based environment. In the earlier workflow, users often had to follow several indirect steps through tables to understand how related wires were grouped. In the present prototype, bundled structures can instead be viewed directly in the graph representation.

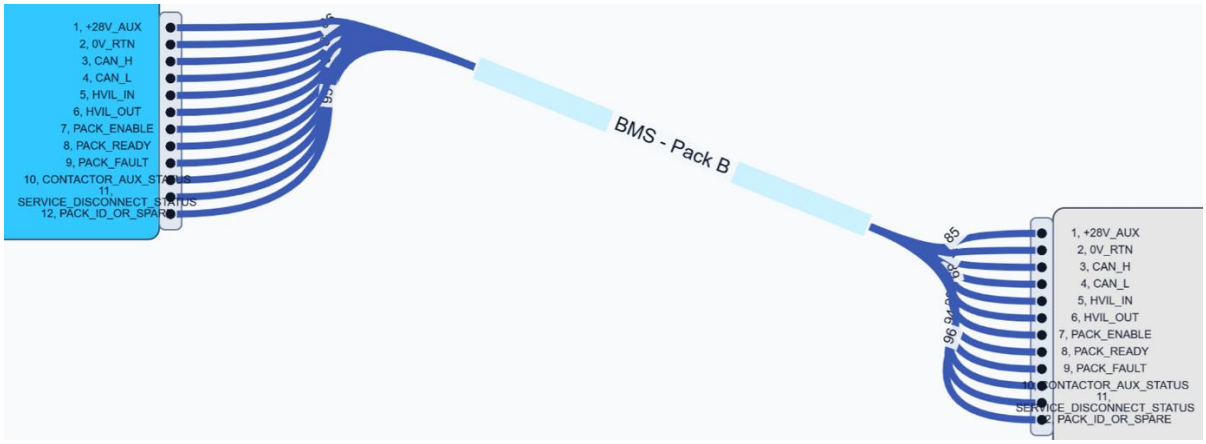


Figure 33 Bus close-up showing bundled wires

User feedback was positive here as well. Users indicated that the visual representation made runs, bundles, and splice-like structures easier to understand than in the earlier tool. The benefit was not only visual. It also reduced the number of interpretation steps needed to understand how related wires belonged together.

Some additional functions relevant to requirement coverage were demonstrated mainly through the implemented system structure rather than through dedicated figures in this section. Export and import support is implemented through structured CSV and XLSX flows, which makes it

possible to bring system data out to Excel-compatible form and later reimport it in a controlled way. The import flow supports several modes and allows the user to control how incoming data should be interpreted. This supports external editing and analysis exchange without forcing all work to remain inside the ESDT.

The ESDT structure also supports future extension and integration. The system is built around a database-backed architecture model with explicit entities for nodes, edges, components, buses, and related objects, together with API-based access from the frontend. Because the ESDT was developed specifically for the studied application, this structure could also be tailored more directly to the workflow than would normally be possible in a fixed licensed environment. This supports the use of one persisted system dataset as a single source of truth and also provides a technical basis for future interfacing with other internal tools. For confidentiality reasons those other systems are not discussed further here, but the implemented structure was chosen specifically so that later integration and extension remain possible.

The compatibility requirement was likewise assessed mainly through the implemented software architecture and deployment approach. The ESDT is browser-based and containerized, with deployment support through Docker and Kubernetes-oriented configuration. In practice this means that the same application stack can be run across different host operating systems without a separate native desktop implementation. The tool was also run successfully on different operating systems during development and testing.

The positive feedback described above also came with several constructive suggestions for further refinement of the workflow and interface. These points are summarised together with the scope of the evaluation in Section 4.7.

4.7 Requirement coverage assessment

Table 4 summarises the requirement coverage assessment for the developed ESDT prototype. The assessment is based on the validation case demonstrated in this section, together with the ESDT functions already described earlier in Section 4.5. For the more objective requirements, the basis is direct functional demonstration in the ESDT. For requirements that depend more strongly on system structure, deployment approach, or extensibility, the basis is the implemented software architecture of the ESDT prototype. For the more subjective requirements, the assessment is also supported by the qualitative user feedback described above.

Table 4 Requirement coverage assessment matrix[1]

Req. ID	Requirement summary	Main evidence / basis	Assessment
1	Store and edit electrical system design data	Demonstrated throughout the case development in Sections 4.5.3 and 4.6.2	Met, directly demonstrated
2	Filter and compile procurement-related component and cabling lists	Table-linked workflow and procurement-related fields shown in Section 4.6.3	Met, directly demonstrated
3	Allow build-related users to understand the system and how to build it	Read-only access, graph view, subsystem filtering, graph-to-table highlighting, installation-related table fields, and supporting user feedback in Section 4.6.3	Met, supported by demonstrated use and user feedback
4	Single source of truth for component and connection attributes	Data model in Section 4.5.2 and linked graph-table behaviour in Section 4.6.3	Met, directly demonstrated
5	Intuitive handling of splice locations and bundling buses	Graph-based bundled connection view, bus handling, and user feedback in Section 4.6.3	Met, supported by demonstrated use and user feedback
6	Display the system so that an uninitiated observer can find their way around easily	Grouping, filtering, colour coding, labels, graph-table linking, and user feedback in Section 4.6.3	Met, supported by demonstrated use and user feedback
7	Bundle data for external editing or analysis	Export and import structure described in Sections 4.5.3 and 4.6.3	Met, based on implemented architecture
8	Backup and undo/redo functions	Demonstrated in the ESDT prototype workflow and described in Sections 4.5.3 and 4.5.4	Met, directly demonstrated
9	Track changes and maintain version history with ability to revert	Version and branch handling described in Section 4.5.4	Met, directly demonstrated
10	Simultaneous access and resolvable conflicting changes	Collaboration and edit blocking described in Section 4.5.4	Met, directly demonstrated
11	Add new attributes or status checklists to components and interconnects	Custom columns and extensibility shown in Sections 4.5.3 and 4.6.3	Met, directly demonstrated
12	Cluster components and interconnects into categories	Grouping and subsystem filtering shown in Section 4.6.3 and supported by user feedback	Met, supported by demonstrated use and user feedback
13	Allow future transition toward a production management tool	Database-backed architecture, API-based access, and structured export/import described in Section 4.6.3	Met, based on implemented architecture
14	Be modular for scaling to larger projects and future database formats	Modular backend and schema-based architecture described in Sections 4.5.2 and 4.5.3	Met, based on implemented architecture
15	Run on Linux, Windows, and macOS	Browser-based and containerized deployment approach described in Sections 4.5 and 4.6.3 (also proven in testing)	Met, demonstrated in prototype context
16	Be intuitive and easy to use without deep programming or database knowledge	General prototype use, graph and table workflow, and user feedback in Section 4.6.3	Met, supported by demonstrated use and user feedback

[1] *Requirement fulfilment is assessed only within the scope of this thesis prototype and demonstration case, not as proof of full operational or industrial readiness.*

4.7.1 Evaluation scope and identified improvement areas

The evaluation presented here is limited to one representative demonstration case and should be understood within that scope. The purpose was not to prove full industrial deployment readiness or to demonstrate superiority over the earlier workflow through a controlled comparative study. The purpose was instead to show that the selected prototype concept can support the intended early-stage system design workflow in a coherent case and can address the requirements that motivated its development.

Some requirements were naturally more direct to demonstrate than others. Functions such as editing, graph-table interaction, subsystem grouping, export-oriented data handling, version history, and simultaneous access could be shown directly in the ESDT prototype. Other requirements, particularly those related to build understanding, orientation for unfamiliar users, and intuitiveness, relied more on qualitative feedback from representative users interacting with the system.

The user feedback was positive overall and supported the conclusion that the new ESDT is a clear improvement over the earlier spreadsheet-based workflow, especially for understanding interconnections, isolating subsystem content, and navigating between visual and tabular representations. At the same time, the evaluation also identified several areas where the workflow can still be improved.

One recurring point was that some parts of the workflow should be streamlined further. The lead electrical systems engineer found the current workflow easier to use than the earlier tabular approach, but still saw value in guiding certain actions more strongly through a more fixed intended workflow. This was especially relevant in areas where the current flexibility also leaves more room for variation in how the user performs a task.

A second point concerned wire routing and visual cleanup. While the current prototype makes it possible to create and inspect detailed interconnections, routing is still largely manual and can become tedious in denser system views. Users also pointed out that bus routing is not yet handled as flexibly as desired, even though bundled structures can already be represented and understood more clearly than in the earlier tool. Related to this, some label placements are still difficult to control and occasional minor visual issues remain in the ESDT prototype. These points mainly affect presentation efficiency and visual clarity rather than the underlying data model, but they are still important for future refinement.

Another thing that was highlighted is a need for more mass-creation and suggestion-based support. The present prototype already includes some tooling for creating larger connector and pin structures more efficiently, and in some places the system can recognise patterns and suggest likely inputs. However, similar support is still desired for other repetitive tasks, particularly for wiring, mating connector creation, and other operations where many similar objects must be entered. Expanding these helper functions would reduce manual work further and make the workflow more efficient in larger use cases.

Within these limits, the electric powertrain validation case still provided a suitable basis for evaluation. It exercised the main workflow steps identified earlier in the thesis, demonstrated that the ESDT could move from early system design definition to more structured connected system data, and showed that the resulting representation is more usable and more flexible than the earlier purely spreadsheet-based approach. The identified shortcomings therefore do not change the overall positive result of the evaluation, but they define important directions for future development and are discussed further in Chapter 5.

5. Analysis and Discussion

This chapter interprets the results in relation to the research gap, the research questions, and the five workflow problems identified earlier.

5.1 Discussion focus and evaluation framing

The evaluation in this thesis was meant to answer a practical question: can the ESDT prototype support a representative early-stage electrical system design workflow in a coherent way? It was not meant to prove industrial readiness, measure productivity gains, or show deployment maturity. The discussion should be read with that scope in mind.

This also matters when reading Table 4. All requirements were assessed as met, but not all in the same way. Some were met through direct prototype demonstration, some were based mainly on implemented architectural support, and some were supported by demonstrated use together with qualitative feedback from a small number of representative users. The result is positive, but it is still a prototype result, not proof of full readiness.

5.2 Addressing the research gap

The gap identified in Chapter 2 was not a lack of methods for modelling systems. The gap was more practical than that. Early-stage electrical system design work still lacks tool support that allows fast definition and restructuring, while also keeping the data structured, shared, versioned, and traceable as the work matures.

The ESDT prototype goes part of the way toward that missing support. In the representative UAV-scale electrified propulsion architecture demonstration case, representing a turbine-generator replacement scenario, the same environment could be used both for rapid early-stage restructuring and for a more detailed and connected system description. That is one of the main points of the work. The tool does not force a jump from whiteboard-like early work to a separate formal system later. It creates a bridge between those stages.

Seen against the neighbouring tools discussed in Chapter 2, the result becomes easier to place. The point is not that tools for modelling, electrical design, or simulation are missing. The point is that they do not line up especially well with the kind of early-stage electrical system design work considered here. The developed ESDT is therefore best understood as a prototype aimed at the gap between flexible early design work and heavier formal engineering tools. Because it was developed specifically for the studied workflow, its data model, views, and interactions could also be tailored more directly to the application than would normally be possible in an existing licensed tool environment.

5.2.1 Relevance in small and agile engineering contexts

The relevance of the research gap becomes especially clear in small and agile engineering contexts such as the one studied in this thesis. In such settings, early system design work often progresses through short iteration cycles, informal discussion, and rapid concept changes rather than through long and highly formalised development chains. This has clear advantages. It allows ideas to be explored quickly, responsibilities to remain close to the technical work, and promising concepts to be revised without heavy process overhead.

At the same time, this way of working also makes certain weaknesses more visible. When a system evolves quickly and much of the detailed system design knowledge sits with a small number of engineers, limitations in version handling, overview, data consistency, and

communication become more difficult to absorb through process alone. The need is therefore not mainly for a heavier modelling environment, but for tool support that fits a faster and more direct way of working while still creating shared, structured, and revisitable system data. In that sense, the gap addressed by this thesis is particularly relevant to modern development teams that need to move quickly without allowing important system knowledge to remain too dependent on local habits, manual duplication, or individual memory.

5.3 Addressing the five identified workflow problems

The five workflow problems provide the most direct way to judge how well the ESDT meets the practical need identified in the thesis.

5.3.1 Collaboration

The difference from the earlier workflow is clear here. In the spreadsheet-based setup, shared work depended heavily on people staying out of each other's way. Although one lead electrical systems engineer carried most of the day-to-day design work, other engineers still needed to access, review, and at times modify parts of the design. In practice, this meant that work could become delayed when one person was editing the file or a certain part of it. In the ESDT, the system design data is shared in one central system and simultaneous work is controlled through locks, access roles, and cell-level edit control. That is a real improvement, because the coordination is no longer mostly social and informal. The tool itself helps prevent conflicts. It also makes it more visible what is editable and what is not.

The move from a mainly tabular representation to a graphical system view also improves collaboration, because it makes interconnections, subsystem boundaries, and design intent easier to understand during review and discussion.

Still, this is not full real-time freeform collaboration. The ESDT uses controlled locking, not unrestricted concurrent editing. That was a sensible choice for this thesis, since the main goal was to keep shared system data coherent. Even so, there is still more to do before this can be called a mature collaborative engineering environment.

5.3.2 Version history and variants

This is one of the strongest parts of the ESDT. In the earlier workflow, different design states were spread across copied files, backups, and notes inside the document. That made it hard to tell what counted as the current baseline, what was only temporary work, and what was an actual alternative.

The ESDT gives these states a clearer structure. The work tree separates saved versions, frozen and unfrozen working versions, and branches. That alone removes a lot of ambiguity that came from file copying and naming habits.

The benefit is mainly traceability and clarity. A branch is treated as a branch, not as another backup file. A saved state is not the same thing as a local working state. That makes the design history easier to follow and easier to discuss.

The limit is just as clear. What is shown here is explicit version handling inside a prototype, not a full configuration management solution. Long-term baseline governance, formal approval flows, and more advanced change management were outside the scope of the thesis.

5.3.3 Large system manageability

The need for a better overview becomes obvious once the demonstration case grows. When interfaces and interconnections are added, the system design becomes dense quickly. That is exactly where the earlier workflow started to struggle. Information was present, but it was hard to see subsystem relations, follow connections, and keep the whole system understandable.

The ESDT improves that in a few direct ways. The graph view gives a clearer picture of structure and connectivity. Grouping and filtering make it easier to isolate parts of the system. The linked table and graph views also help, since a user can move between overview and detail without switching to disconnected data.

The feedback points in the same direction. Users found subsystem isolation, connection tracing, and explanation of the system easier in the grouped graph view. That matches the intended role of the tool quite well.

But this area is not finished. The evaluation also showed where the tool still becomes heavy to use, especially with wire routing, bus routing, label placement, and general layout cleanup. Those issues matter because once the system grows, visual clarity is not a minor detail. It is part of whether the workflow stays usable at all.

5.3.4 Single source of truth and data consistency

This is probably the most important result in the thesis. The earlier workflow had no strong single source of truth. The same information could appear in several places and had to be kept aligned manually. Connector-related part information is one example, but the problem was broader than that. Once the same fact is written more than once, inconsistency becomes a normal risk.

The ESDT changes that structure. The system design is stored once in a shared persisted model, and the graph and table views read from that same model. So the user is no longer maintaining several semi-connected representations of the system. There is one system design, shown in different ways.

That does not solve every future integration issue. It does, however, remove a basic weakness of the earlier workflow. The ESDT makes it much harder to create inconsistency through repeated manual entry, disconnected tables, or unclear version context. For the problem defined in this thesis, that is a substantial step forward.

5.3.5 Usability for new users

The earlier workflow was not necessarily difficult for new users in the sense of basic tool use, since most engineers are already familiar with spreadsheet-based work. The harder part was gaining a clear overview of the electrical system itself. As the system grew, understanding how parts were connected, how subsystems related to each other, and how the overall design was organised depended increasingly on careful tracing through tabular data and local working habits. This made handover and broader system understanding more difficult.

The ESDT improves this mainly through its graphical system view. The graph view gives a more direct picture of the system, especially when grouping and filtering are used to isolate relevant parts. The user feedback supports this. People found it easier to explain the system, easier to trace how parts were connected, and easier to orient themselves in the model.

That said, the evidence here is more limited than for the harder functional points. The feedback was relevant, but it was still small and informal. This was not a formal usability study. The tool

also still leaves some workflow choices open, which can be useful for experienced users but can also make the process less guided for someone new. So the result here is promising, but it should be read with some caution.

5.4 Relation to literature

The results align with the five literature areas in Chapter 2, and the clearest links are practical rather than theoretical.

For conceptual electrical system design tools, the thesis shows the value of a tool that supports actual system design work, not only formal description. The ESDT deals directly with components, interfaces, connections, and restructuring in one working environment.

For MBSE and system design modelling, the link is the use of one shared model, explicit relations, and multiple consistent views. The ESDT is lighter than a full formal MBSE environment, but it follows the same basic logic where it matters here.

For graph-based knowledge representation, the fit is direct. The demonstration case showed why connected system data is easier to understand in a graph-based view than in disconnected lists alone.

For design data management and versioning, the alignment is one of the clearest in the thesis. Shared data, explicit version states, and reduced dependence on copied files speak directly to the problems highlighted in that literature.

For decision support under uncertainty, the contribution is more indirect. The ESDT is not a trade study tool by itself, but it creates a cleaner basis for comparing alternatives and revisiting earlier design states.

The comparison with neighbouring tools also supports the picture that emerged from the literature review: the challenge is not mainly a lack of methods, but a weaker connection between those methods and the practical needs of early-stage electrical system design work.

5.5 Limitations and validity considerations

The evaluation has clear limits, and they matter for how far the conclusions can be taken.

First, the work was tested in one demonstration case. The case was suitable for the thesis because it was large enough to exercise the core workflow and relevant enough to represent the intended domain. Still, one case is one case. It does not show that the same result would hold across all aircraft electrical system designs or all team settings.

Second, the user feedback was small and informal. It helped show how the ESDT was perceived by representative users, and it was useful for judging things like overview and ease of explanation. But it was not a formal usability study, and it should not be presented as one.

Third, several areas are still at prototype level even if they work well enough for the evaluation. That is especially true for collaboration, version governance, and long-term data control. The ESDT shows controlled locking and explicit version handling, but it does not show a full industrial process around review, approval, release, and long-term governance.

Fourth, the thesis does not prove time savings, lower error rates, or improved long-term engineering efficiency. Those things may well be influenced by the tool design, but they were not measured here. The safer claim is the one supported by the results: several typical inconsistency

mechanisms are reduced by design because the system design is shared, versioned explicitly, and not maintained through repeated manual duplication.

Finally, Table 4 should not be read too strongly. "Met" does not mean "completely solved in every possible sense". It means the requirement was addressed to a sufficient level within this evaluation, with different types of support behind that judgement: direct demonstration, demonstrated use together with feedback, or implemented architectural support.

5.6 Future work

Several areas in the ESDT prototype would benefit from further development, especially where the evaluation exposed clear remaining limits.

One area is workflow guidance. The ESDT prototype already supports the main tasks, but some parts still rely too much on the user knowing how to proceed. Better guidance would help most with usability for new users and with more consistent working practice. That connects mainly to Problem 5 and Requirement 16.

Another area is layout and routing. Better wire and bus routing, label handling, and clearer automatic arrangement would improve readability once the system becomes large. This connects directly to Problem 3 and Requirement 6.

There is also room for more mass-creation and suggestion-based support. Repeated tasks such as connector definition, mating connector creation, and recurring wiring patterns should not need as much manual work as they do now. That would mainly help Problems 3 and 5, and it would strengthen Requirements 11 and 16.

Collaboration and version handling should also be developed further. The current mechanisms are enough to show the concept, but future versions should support stronger review flows, clearer change summaries, better baseline handling, and more mature governance of shared work. That links mainly to Problems 1 and 2, and to Requirements 9 and 10.

The evaluation itself should be expanded as well. More cases, more users, and longer use in real engineering work would give a much stronger basis for judging workflow impact, scalability, and usability over time.

Lastly, the tool should be pushed further toward downstream use. The single-source-of-truth idea becomes more valuable if the same system design data can feed later analysis and implementation-related work without losing structure. That mainly connects to Problem 4 and to Requirements 7, 13, and 14.

5.7 Ethical aspects

The ethical aspects of this thesis are linked mainly to responsible handling of engineering information rather than to formal human-subject issues. In complex technical systems, unclear, duplicated, or inconsistent system design data can influence later design decisions in ways that are difficult to detect. From that perspective, traceability, explicit version context, and a clearer single source of truth are not only technical improvements but also ethically relevant, since they reduce the risk of avoidable misunderstanding and error in collaborative engineering work.

The ESDT prototype should not be overinterpreted. The thesis does not show that the tool guarantees correct system designs, safety, or industrial readiness. It only demonstrates that the workflow can be made more coherent and that several common inconsistency mechanisms can

be reduced by design. Ethical use of such a tool therefore still requires engineering judgement, review, and awareness of the limits of prototype-level evidence.

There is also an ethical aspect in accessibility of engineering work. When important workflow knowledge is hidden in local habits or informal experience, new users become more dependent on specific individuals and handover becomes more difficult. A tool that makes the system structure and relations easier to understand can therefore support a more transparent and accessible engineering process.

5.8 Economic, environmental, and societal sustainability aspects

The sustainability relevance of this thesis should be understood mainly in relation to the broader development of future electric and hybrid-electric aircraft, rather than in relation to the software prototype alone. In that sense, the work can be related loosely to Agenda 2030, the United Nations framework for sustainable development and its 17 Sustainable Development Goals, although it is not analysed formally through that framework. The developed ESDT is relevant primarily as process support in the early development of future electric and hybrid-electric aircraft, where propulsion-system alternatives, power-distribution concepts, and supporting subsystems are still being defined, revised, tested, and communicated.

5.8.1 Economic sustainability

From an economic perspective, the main relevance of the work lies in improving how early-stage electrical systems are developed and compared. In future electric and hybrid-electric aircraft development, architectural choices around energy storage, power conversion, distribution, control, and protection can influence later integration effort, procurement planning, redesign needs, and overall engineering cost. A tool that makes those system designs easier to structure, revise, and communicate can therefore help reduce wasted effort before more resource-intensive downstream commitments are made. This is especially relevant in electric and hybrid-electric aircraft development, where many concepts must be matured quickly enough to support continued build, integration, and testing without excessive redesign effort.

In the industrial context studied, the previous spreadsheet-based workflow created repeated manual entry, weak version visibility, and additional coordination overhead. By moving toward one structured and shared system design model, the ESDT provides a basis for reducing such inefficiencies and for making concept maturation less dependent on local workarounds. The economic contribution is therefore not that the thesis proves direct cost savings, but that it demonstrates a workflow approach with clearer potential to support more efficient early development of electrified aircraft systems.

5.8.2 Environmental sustainability

The environmental relevance of the thesis is indirect but important because it relates to the development process for future electric and hybrid-electric aircraft. Such aircraft concepts depend on repeated early-stage iteration of batteries, converters, distribution structures, protection strategies, control interfaces, and supporting subsystems before later sizing and performance work can be carried out with confidence. A tool that supports clearer system definition and restructuring can therefore help engineers assess, document, and revise candidate system arrangements more coherently at an earlier stage, before later sizing, integration, and performance evaluation.

The ESDT does not reduce emissions by itself and it is not a simulation tool for proving environmental performance. Its environmental value lies instead in supporting a development

workflow in which alternatives can be structured, revised, and carried forward with less duplication and less risk of losing design intent. Over time, that can provide a better basis for later analyses related to mass, efficiency, integration, and other environmentally relevant characteristics of electric and hybrid-electric aircraft concepts.

5.8.3 Societal sustainability

The societal relevance of the work also comes mainly through the development context it supports. Future electric and hybrid-electric aircraft are being pursued in part because aviation faces strong societal pressure to reduce environmental impact while maintaining useful air transport capability. In that context, the quality and speed of early engineering work matters, because unclear system design information can slow development, complicate collaboration, and make important design decisions harder to review across disciplines. In the longer term, clearer and faster development of such systems can support the wider introduction of lower-emission aircraft concepts in applications where aviation still provides important societal value.

In aerospace-related development, and especially in safety-relevant domains, it is valuable that system information can be understood not only by the primary designer but also by colleagues involved in integration, procurement, build preparation, and later review. The ESDT supports this by making system structure, interfaces, and interconnections more visible and easier to follow than in the earlier spreadsheet-based workflow. Its societal contribution is therefore not a direct end-user effect, but support for a more transparent and communicable engineering process in a safety-relevant domain where technical decisions can influence how future lower-emission aircraft systems are developed, reviewed, and eventually applied.

6. Conclusions

This chapter gathers the main answers from the thesis and states the overall conclusion within the scope of what was actually demonstrated.

6.1 Answers to the research questions

The main research question can be answered in a fairly direct way. In the industrial context studied at The Rainforest, Saab's innovation hub, a dedicated electrical system design tool can provide a more coherent early-stage workflow than the earlier spreadsheet-based approach, provided that it combines graph-based restructuring, shared structured data, linked detail views, explicit version handling, and controlled multi-user access. The ESDT prototype demonstrated this in a representative UAV-scale electrified propulsion architecture case from the Saab Rainforest context, reflecting early design-stage work relevant to future electric and hybrid-electric aircraft systems. The ESDT developed in this thesis is not a simulation environment, but a prototype that can serve as a basis for further implementation of the intended tool. It provides a more structured design basis for later sizing, analysis, testing, and more detailed design work.

The first sub question asked what the main problems of the earlier workflow were. Five were identified: limited collaboration, weak handling of versions and variants, poor overview in larger systems, lack of a strong single source of truth, and a high entry threshold for new users.

The second sub question asked what information the tool must manage and how it should be structured. The answer is that components, interfaces, connections, connector-related information, grouping structures, custom attributes, and version context need to be handled as linked structured data rather than repeated values across disconnected sheets.

The third sub question asked which functions and views are needed. The thesis showed that the core combination is a graph view for system structure and restructuring, table views for dense data work, and support around them for filtering, grouping, undo and redo, version handling, shared access, extensibility, and structured import and export.

The fourth sub question asked what the demonstration case actually showed. It showed that the ESDT prototype could support the intended workflow from initial definition to a more detailed connected system design, and that it reduced several common inconsistency mechanisms by design through one shared model, linked views, explicit version states, and controlled shared editing.

6.2 Main contributions

The thesis makes three main contributions, together with a broader practical positioning of the problem.

The first contribution is the full chain from observed workflow problems to a working prototype. The thesis did not stop at identifying weaknesses in the earlier approach. It translated them into requirements, compared solution principles, selected a concept, implemented it, and tested it in a representative case.

The second contribution is a clearer practical framing of the research gap. The thesis suggests that the missing piece is not mainly another modelling method. The bigger need is support for the stage where design work still needs to be quick and flexible, but where the information should already be structured, shared, and possible to follow over time.

The third contribution is the demonstrated result itself. Within the limits of a master's thesis, the work shows that early-stage electrical system design can be handled in a way that is more structured, easier to navigate, and less dependent on manual spreadsheet synchronisation than before. In that sense, the developed ESDT can be understood as an exploration of a lower-overhead bridge between spreadsheet-based work and heavier engineering environments.

A further contribution of the thesis is a clearer practical positioning of the problem and the developed ESDT. The work shows that the relevant gap is not simply a lack of methods for system modelling or electrical analysis. Instead, the gap lies in practical support for early-stage electrical system design work that must remain fast and flexible while still becoming structured, shared, and traceable. In this sense, the developed ESDT contributes an exploration of a lower-overhead bridge between manually maintained spreadsheet work and heavier MBSE or electrical engineering environments.

6.3 Final conclusion

Taken together, the findings of this thesis indicate that the identified workflow gap is real and can be addressed through a practical prototype tool tailored to early-stage electrical system design work. In the studied industrial setting, the ESDT prototype showed that graphical restructuring, structured shared data, linked graph and table views, and explicit version handling can be combined in one environment that supports system definition, revision, and communication more coherently than the earlier spreadsheet-based workflow.

The work does not demonstrate industrial readiness, full downstream integration, or replacement of later simulation and detailed electrical design environments. Nevertheless, it has shown that exploratory early system design work for future electric and hybrid-electric aircraft can be brought into a more structured, shared, and reusable form without forcing an immediate transition to heavier engineering environments. The main conclusion is therefore that a lower-overhead but structured tool approach is both relevant and technically credible for this stage of development.

6.4 Future work

Several directions for future work follow from the present thesis. First, the ESDT prototype should be evaluated in additional cases, with a broader user group and over longer periods of practical use, in order to assess workflow impact more robustly. Second, the interface and workflow should be refined further, particularly in relation to routing support, layout clarity, guidance for less experienced users, and efficient handling of larger systems. Third, the structured system model should be connected more clearly to later analysis, implementation, and other engineering environments, so that the single-source-of-truth principle can support a longer part of the development chain. These directions are discussed in more detail in Section 5.6.

Further work is therefore required not only to mature the ESDT itself, but also to test how far this type of tool can support the wider transition from exploratory system design work toward more integrated engineering practice in future electric and hybrid-electric aircraft development.

7. Bibliography

[1] M. Lindberg and J. Leijon, 'Electrifying aviation: Innovations and challenges in airport electrification for sustainable flight', *Adv. Appl. Energy*, vol. 18, p. 100222, Jun. 2025, doi: 10.1016/j.adapen.2025.100222.

[2] 'History | About | Saab', Start. Accessed: Mar. 20, 2026. [Online]. Available: https://www.saab.com/about/company-in-brief/history

[3] 'Saab | Keeping people and society safe', Start. Accessed: Mar. 20, 2026. [Online]. Available: https://www.saab.com/

[4] 'The Rainforest', The Rainforest. Accessed: Mar. 20, 2026. [Online]. Available: https://thernfrst.io/

[5] SAAB (Redacted), '3D render the Rainforests autonomous UAV with section cut'. The Rainforest, Linköping, Sweden, 04 2026.

[6] 'Cameo Systems Modeler', Dassault Systèmes. Accessed: Mar. 20, 2026. [Online]. Available: https://www.3ds.com/products/catia/no-magic/cameo-systems-modeler

[7] 'Capella MBSE Tool - Arcadia'. Accessed: Mar. 20, 2026. [Online]. Available: https://mbse-capella.org/arcadia.html

[8] 'Capital systems architect', Siemens Digital Industries Software. Accessed: Mar. 20, 2026. [Online]. Available: https://www.siemens.com/en-us/products/capital/offerings/systems-architect/

[9] 'Eplan Electric P8: Software for electrical engineering and schematic design'. Accessed: Mar. 20, 2026. [Online]. Available: https://www.eplan.com/us-en/products/eplan-electric-p8/

[10] 'E3.series', English. Accessed: Mar. 20, 2026. [Online]. Available: https://www.zuken.com/en/product/e3series/

[11] 'Download LTspice | Analog Devices'. Accessed: Mar. 20, 2026. [Online]. Available: https://www.analog.com/en/resources/design-tools-and-calculators/ltspice-simulator.html

[12] S. Liscouët-Hanke, J.-C. Maré, and S. Pufe, 'Simulation Framework for Aircraft Power System Architecting', *J. Aircr.*, vol. 46, no. 4, pp. 1375–1380, Jul. 2009, doi: 10.2514/1.41304.

[13] G. F. Esdras and S. Liscouet-Hanke, 'An Electrical Load Estimation Tool for Aircraft Conceptual Design', presented at the SAE 2013 AeroTech Congress & Exhibition, Montréal, Canada, Sep. 2013, pp. 2013-01–2206. doi: 10.4271/2013-01-2206.

[14] A. G. Garriga, P. Govindaraju, S. S. Ponnusamy, N. Cimmino, and L. Mainini, 'A modelling framework to support power architecture trade-off studies for More-Electric Aircraft', *Transp. Res. Procedia*, vol. 29, pp. 146–156, 2018, doi: 10.1016/j.trpro.2018.02.013.

[15] S. Diampovesa, A. Hubert, and P.-A. Yvars, 'Designing physical systems through a model-based synthesis approach. Example of a Li-ion battery for electrical vehicles', *Comput. Ind.*, vol. 129, p. 103440, Aug. 2021, doi: 10.1016/j.compind.2021.103440.

[16] S. Subarna, A. K. Jawale, A. S. Vidap, S. D. Sadachar, S. Fliginger, and S. Myla, 'Using a Model Based Systems Engineering Approach for Aerospace System Requirements Management', in *2020 AIAA/IEEE 39th Digital Avionics Systems Conference (DASC)*, San Antonio, TX, USA: IEEE, Oct. 2020, pp. 1–8. doi: 10.1109/DASC50938.2020.9256589.

[17] B. Friedland, R. Malone, and J. Herrold, 'Systems Engineering a Model Based Systems Engineering Tool Suite: The Boeing Approach', *INCOSE Int. Symp.*, vol. 26, no. 1, pp. 386–398, Jul. 2016, doi: 10.1002/j.2334-5837.2016.00167.x.

[18] A. K. Jeyaraj, N. Tabesh, and S. Liscouet-Hanke, 'Connecting Model-based Systems Engineering and Multidisciplinary Design Analysis and Optimization for Aircraft Systems Architecting', in *AIAA AVIATION 2021 FORUM*, VIRTUAL EVENT: American Institute of Aeronautics and Astronautics, Aug. 2021. doi: 10.2514/6.2021-3077.

[19] S. Gao, W. Cao, L. Fan, and J. Liu, 'MBSE for Satellite Communication System Architecting', *IEEE Access*, vol. 7, pp. 164051–164067, 2019, doi: 10.1109/ACCESS.2019.2952889.

[20] A. Salado and P. Wach, 'Constructing True Model-Based Requirements in SysML', *Systems*, vol. 7, no. 2, p. 19, Mar. 2019, doi: 10.3390/systems7020019.
[21] S. Tan, S. Cimtalay, and D. Mavris, 'An MBSE Approach to Hydrogen Combustion Turbofan Propulsion System Design', in *AIAA AVIATION FORUM AND ASCEND 2024*, Las Vegas, Nevada: American Institute of Aeronautics and Astronautics, Jul. 2024. doi: 10.2514/6.2024-4378.
[22] L. Wang, M. Izygon, S. Okon, H. Wagner, and L. Garner, 'Effort to Accelerate MBSE Adoption and Usage at JSC', in *AIAA SPACE 2016*, Long Beach, California: American Institute of Aeronautics and Astronautics, Sep. 2016. doi: 10.2514/6.2016-5542.
[23] J. A. Zhang, B. Bagdatli, and D. N. Mavris, 'Leveraging SysML V2 for Integration of MBSE and Multidisciplinary System Development', in *AIAA SCITECH 2023 Forum*, National Harbor, MD & Online: American Institute of Aeronautics and Astronautics, Jan. 2023. doi: 10.2514/6.2023-1895.
[24] M. Dong, G. Wang, J. Lu, S. Wu, and J. Ma, 'Ontology formalism and semantic rules supporting traceability management in model-based systems engineering', *J. Eng. Des.*, pp. 1–26, Jul. 2025, doi: 10.1080/09544828.2025.2532291.
[25] S. Glinski *et al.*, 'An MBSE Framework to Identify Regulatory Gaps for Electrified Transport Aircraft', in *2022 IEEE Transportation Electrification Conference & Expo (ITEC)*, Anaheim, CA, USA: IEEE, Jun. 2022, pp. 772–777. doi: 10.1109/ITEC53557.2022.9813806.
[26] S. Datta, R. Roy, M. V. Bendarkar, E. Harrison, and D. N. Mavris, 'MBSE-Enabled Risk Reduction for Certification of Novel Aircraft Configurations', in *AIAA SCITECH 2022 Forum*, San Diego, CA & Virtual: American Institute of Aeronautics and Astronautics, Jan. 2022. doi: 10.2514/6.2022-1189.
[27] T. R. Browning, 'Applying the design structure matrix to system decomposition and integration problems: a review and new directions', *IEEE Trans. Eng. Manag.*, vol. 48, no. 3, pp. 292–306, Aug. 2001, doi: 10.1109/17.946528.
[28] T. R. Browning, 'Design Structure Matrix Extensions and Innovations: A Survey and New Opportunities', *IEEE Trans. Eng. Manag.*, vol. 63, no. 1, pp. 27–52, Feb. 2016, doi: 10.1109/TEM.2015.2491283.
[29] J. Schmidt and S. Rudolph, 'Graph-Based Design Languages: A Lingua Franca for Product Design Including Abstract Geometry', *IEEE Comput. Graph. Appl.*, vol. 36, no. 5, pp. 88–93, Sep. 2016, doi: 10.1109/MCG.2016.89.
[30] D. R. Herber, J. T. Allison, R. Buettner, P. Abolmoali, and S. S. Patnaik, 'Architecture Generation and Performance Evaluation of Aircraft Thermal Management Systems Through Graph-based Techniques', in *AIAA Scitech 2020 Forum*, Orlando, FL: American Institute of Aeronautics and Astronautics, Jan. 2020. doi: 10.2514/6.2020-0159.
[31] N. Nassar and M. Austin, 'Model-Based Systems Engineering Design and Trade-Off Analysis with RDF Graphs', *Procedia Comput. Sci.*, vol. 16, pp. 216–225, 2013, doi: 10.1016/j.procs.2013.01.023.
[32] J. Lu, J. Ma, X. Zheng, G. Wang, H. Li, and D. Kiritsis, 'Design Ontology Supporting Model-Based Systems Engineering Formalisms', *IEEE Syst. J.*, vol. 16, no. 4, pp. 5465–5476, Dec. 2022, doi: 10.1109/JSYST.2021.3106195.
[33] A. Zech, R. Stetter, S. Rudolph, and M. Till, 'Capturing the Design Rationale in Model-Based Systems Engineering of Geo-Stations', *Proc. Des. Soc.*, vol. 2, pp. 2015–2024, May 2022, doi: 10.1017/pds.2022.204.
[34] J. Haiar, J. Lewis, and H. Tiedeman, 'Model-Based Engineering for Platform System Design', in *2006 ieee/aiaa 25TH Digital Avionics Systems Conference*, Portland, OR, USA: IEEE, Oct. 2006, pp. 1–12. doi: 10.1109/DASC.2006.313728.
[35] Y. Uludağ, Ö. Bayoğlu, B. Candan, and H. Yılmaz, 'Managing Requirement Development Challenges in IMA Platform Development', in *2024 AIAA DATC/IEEE 43rd Digital Avionics Systems Conference (DASC)*, San Diego, CA, USA: IEEE, Sep. 2024, pp. 1–10. doi: 10.1109/DASC62030.2024.10749489.

[36] Z. Xu and S. Yang, 'Research and Application of the Data Model System in MBSE', *IEEE Access*, vol. 13, pp. 195167–195186, 2025, doi: 10.1109/ACCESS.2025.3632351.
[37] V. T and V. Hegde, 'Design and Development of Automotive Domain Needs Using Rhapsody Model Manager', in *2023 7th International Conference on Computation System and Information Technology for Sustainable Solutions (CSITSS)*, Bangalore, India: IEEE, Nov. 2023, pp. 1–6. doi: 10.1109/CSITSS60515.2023.10334143.
[38] S. Wu *et al.*, 'Cognitive Digital Thread Tool-Chain for Model Versioning in Model-Based Systems Engineering', 2025, *SSRN*. doi: 10.2139/ssrn.5129653.
[39] T. Kurtoglu, I. Y. Tumer, and D. C. Jensen, 'A functional failure reasoning methodology for evaluation of conceptual system architectures', *Res. Eng. Des.*, vol. 21, no. 4, pp. 209–234, Oct. 2010, doi: 10.1007/s00163-010-0086-1.
[40] L. W. T. Ng and K. E. Willcox, 'Monte Carlo Information-Reuse Approach to Aircraft Conceptual Design Optimization Under Uncertainty', *J. Aircr.*, vol. 53, no. 2, pp. 427–438, Mar. 2016, doi: 10.2514/1.C033352.
[41] G. Cinar, Y. Cai, M. V. Bendarkar, R. K. Denney, and D. N. Mavris, 'System Analysis and Design Space Exploration of Regional Aircraft with Electrified Powertrains', in *AIAA SCITECH 2022 Forum*, San Diego, CA & Virtual: American Institute of Aeronautics and Astronautics, Jan. 2022. doi: 10.2514/6.2022-1994.
[42] T. Tanimura and K. Rinoie, 'Consideration of Future Development of Hybrid Electric Aircraft using Epoch-Era Analysis', in *AIAA Scitech 2021 Forum*, VIRTUAL EVENT: American Institute of Aeronautics and Astronautics, Jan. 2021. doi: 10.2514/6.2021-1639.
[43] M. J. Daskilewicz, B. J. German, T. T. Takahashi, S. Donovan, and A. Shajanian, 'Effects of disciplinary uncertainty on multi-objective optimization in aircraft conceptual design', *Struct. Multidiscip. Optim.*, vol. 44, no. 6, pp. 831–846, Dec. 2011, doi: 10.1007/s00158-011-0673-4.
[44] A. Gazaix *et al.*, 'Investigation of Multi-Disciplinary Optimisation for Aircraft Preliminary Design', presented at the Aerospace Technology Conference and Exposition, Toulouse, France, Oct. 2011, pp. 2011-01–2761. doi: 10.4271/2011-01-2761.
[45] 'Teamwork Cloud', Dassault Systèmes. Accessed: Mar. 20, 2026. [Online]. Available: https://www.3ds.com/products/catia/no-magic/teamwork-cloud
[46] 'Capella MBSE Tool - Features'. Accessed: Mar. 20, 2026. [Online]. Available: https://mbse-capella.org/features.html
[47] 'E3.FunctionalDesign', English. Accessed: Mar. 20, 2026. [Online]. Available: https://www.zuken.com/en/product/e3series/functional-design/
[48] 'Eplan Preplanning: Digitalized planning – right from the start'. Accessed: Mar. 20, 2026. [Online]. Available: https://www.eplan.com/us-en/products/eplan-preplanning/
[49] 'System Composer'. Accessed: Mar. 20, 2026. [Online]. Available: https://se.mathworks.com/products/system-composer.html

8. Appendix

8.1 AI usage statement

Generative AI tools were used in a limited supporting role during this thesis. Their use was restricted mainly to proofreading, feedback, and the suggestion of alternative formulations during revision of the author's own text. AI was also used to generate icons for the morphological chart to improve the visual presentation of ideas. In addition, AI coding tools were used to support the development of the ESDT prototype software tool, particularly by accelerating parts of the implementation and debugging process. The tools used included OpenAI GPT-5 series models, OpenAI Codex, and Anthropic Claude Code. No AI tool was used to generate research results, fabricate references, or make final scientific judgments without verification by the author. All final text, technical decisions, analysis, interpretations, and conclusions presented in this thesis remain the responsibility of the author.